

Module No: 1 Introduction to Algorithm And Flowchart

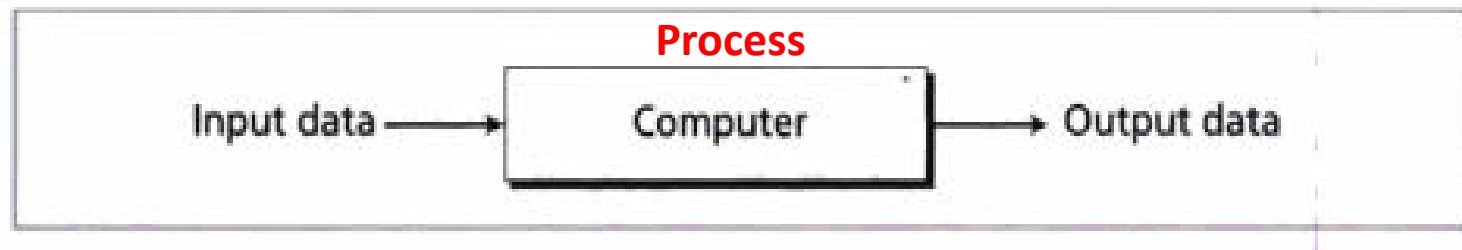
Lecture No: 1

Turing Model, Von Neumann Model and Types of Programming Languages



What is a Computer?

- A computer is a programmable electronic device that accepts raw data as input and **processes it with a set of instructions (a program)** to produce the result as output.



- A computer is designed to execute applications and provides a variety of solutions through integrated hardware and software components.

Program

- Computer works with the help of programs and data.
- A program is a **set of logically related instructions** that is arranged in a **sequence** that directs the computer in solving a problem tells the computer what to do with data.

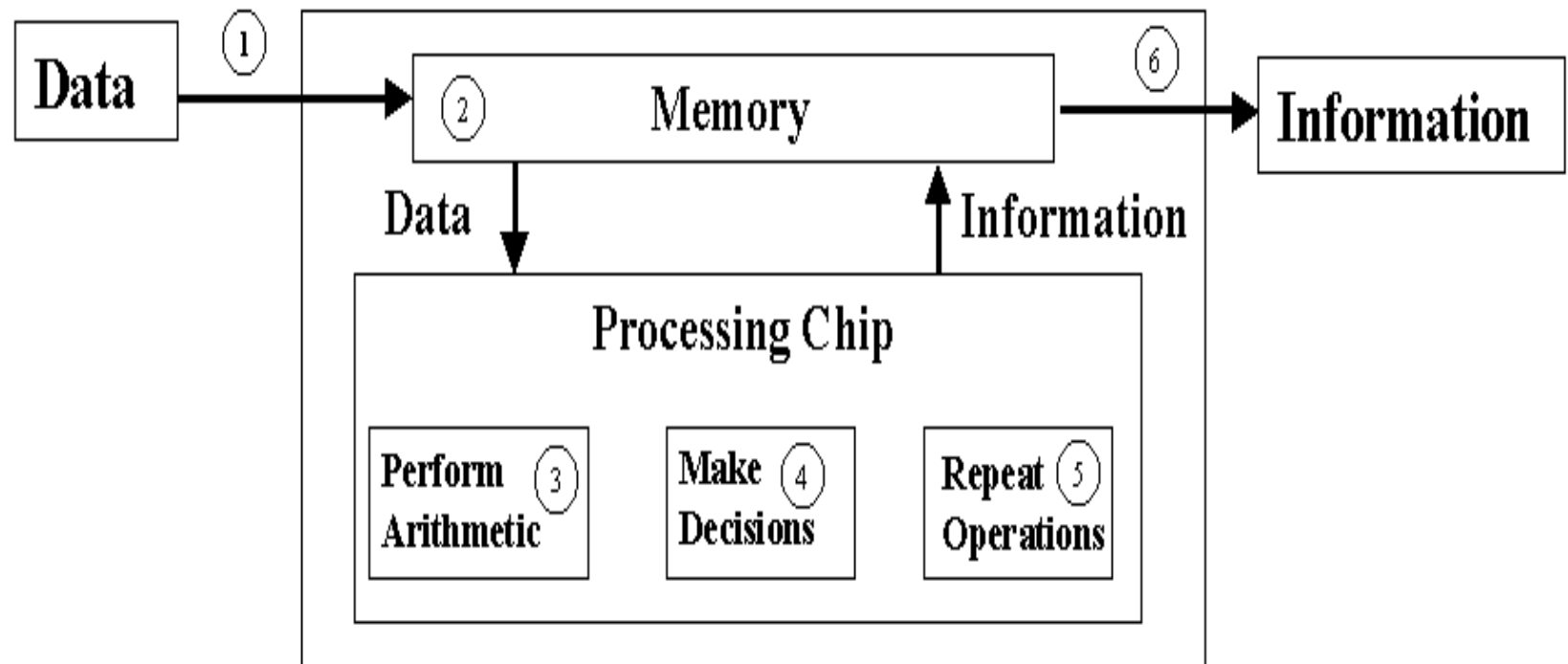


Six Basic Computer Operations

1. A computer can receive (input) data
2. A computer can store data in memory
3. A computer can perform arithmetic and manipulate text strings
4. A computer can compare the contents of two memory locations and select one of two alternatives
5. A computer can repeat a group of operations
6. A computer can output information (processed data)



Computer Operations



Computational Model

- The common basis of programming language and computer architecture is known as Computational Model.
- Provides higher level of abstraction(general idea) than the programming language and the computer architecture
- It tells us how computer functions or runs.
- A **computational model** uses computer programs to simulate and study complex systems using an algorithmic or mechanistic approach and is widely used in a diverse range of fields.
- **Basic Computational Models are :**
 - Turing
 - Von Neumann



Computational Model

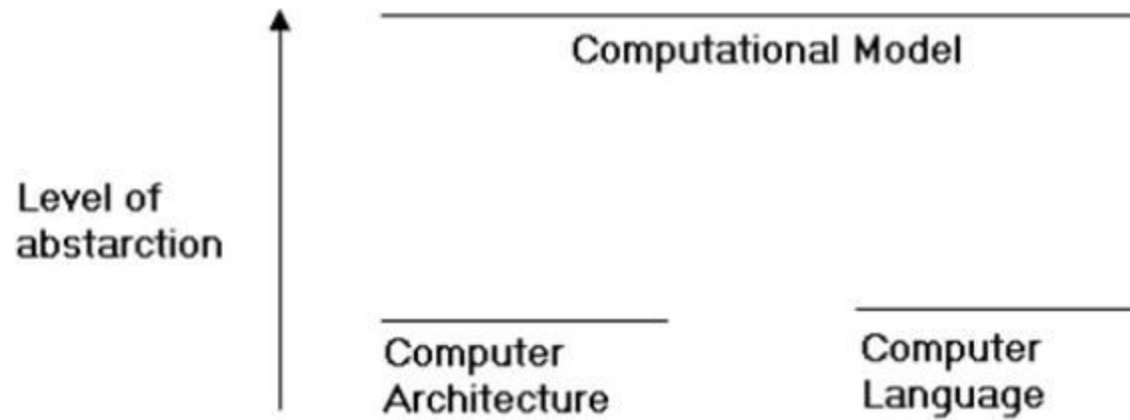
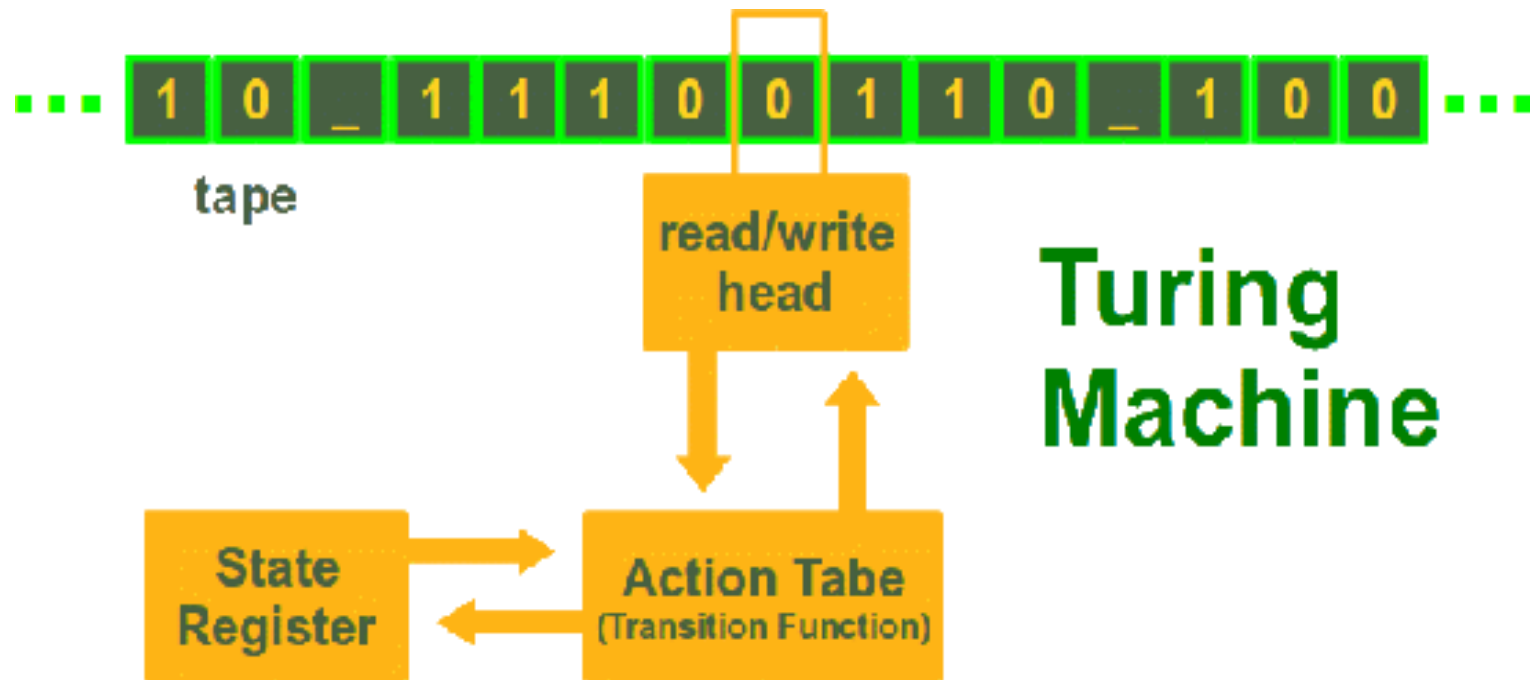


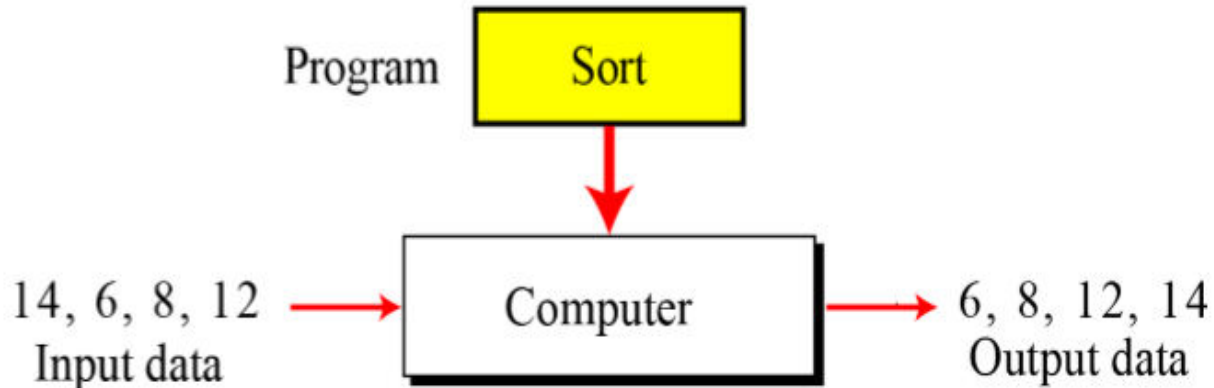
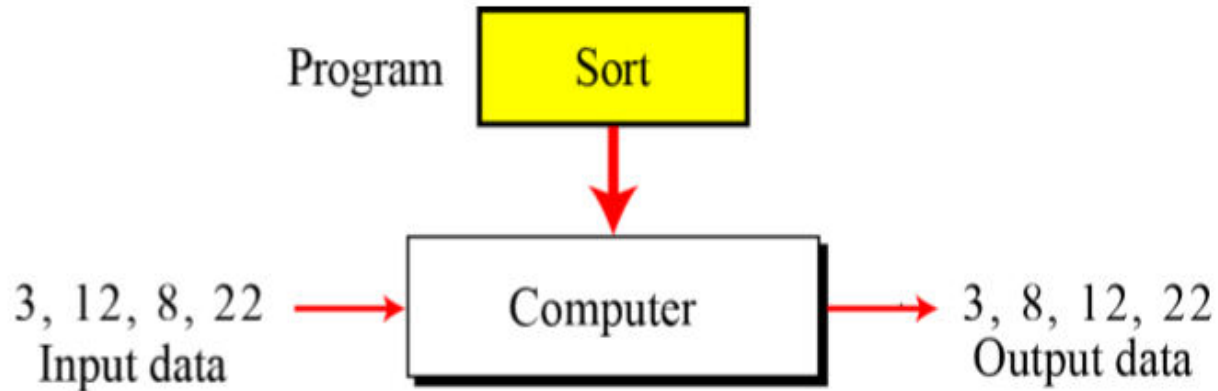
Figure 1.1

Turing Model

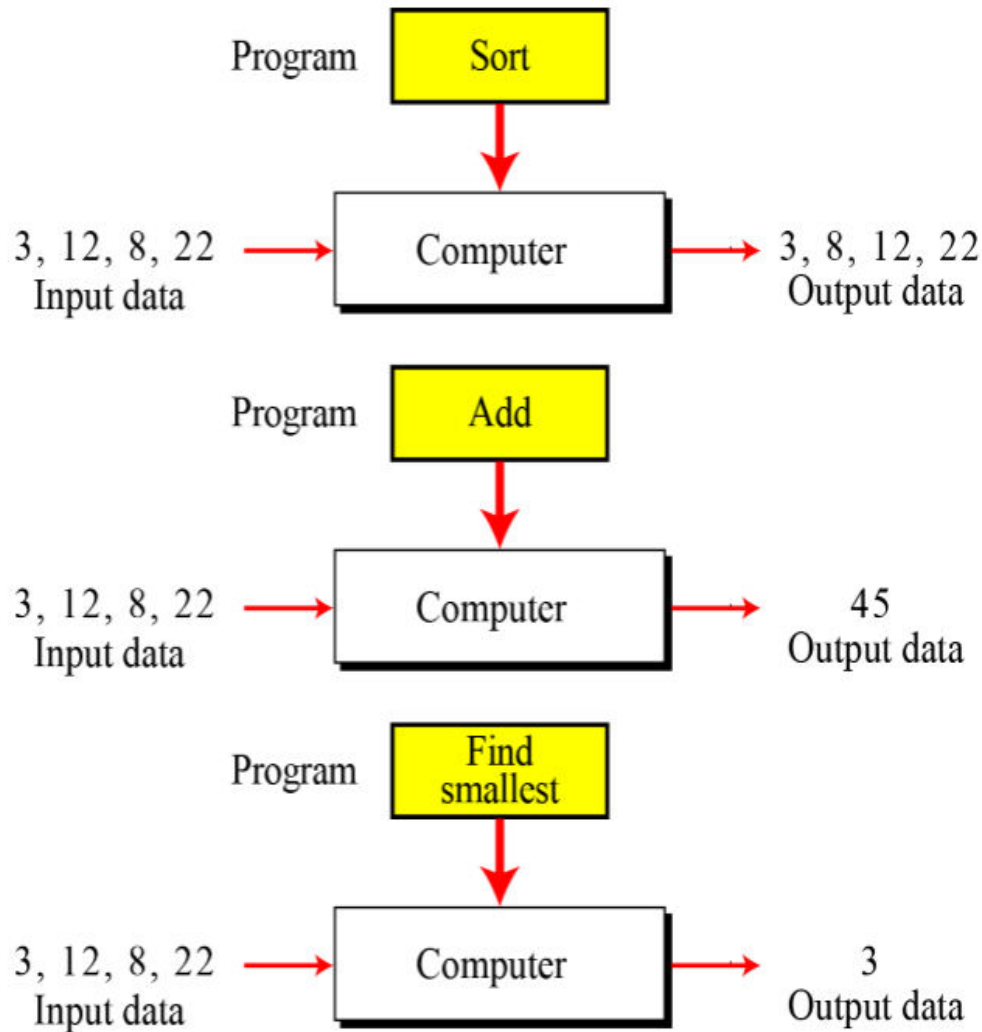
- Alan Turing designed the model which is an abstract machine that processed the symbols written on a strip of tape as the rules of the symbols defined in the table, which defines the mathematical model of the computer.
- The Turing model is a fundamental machine model based on which modern, general-purpose computer are designed.



Example :



Example :

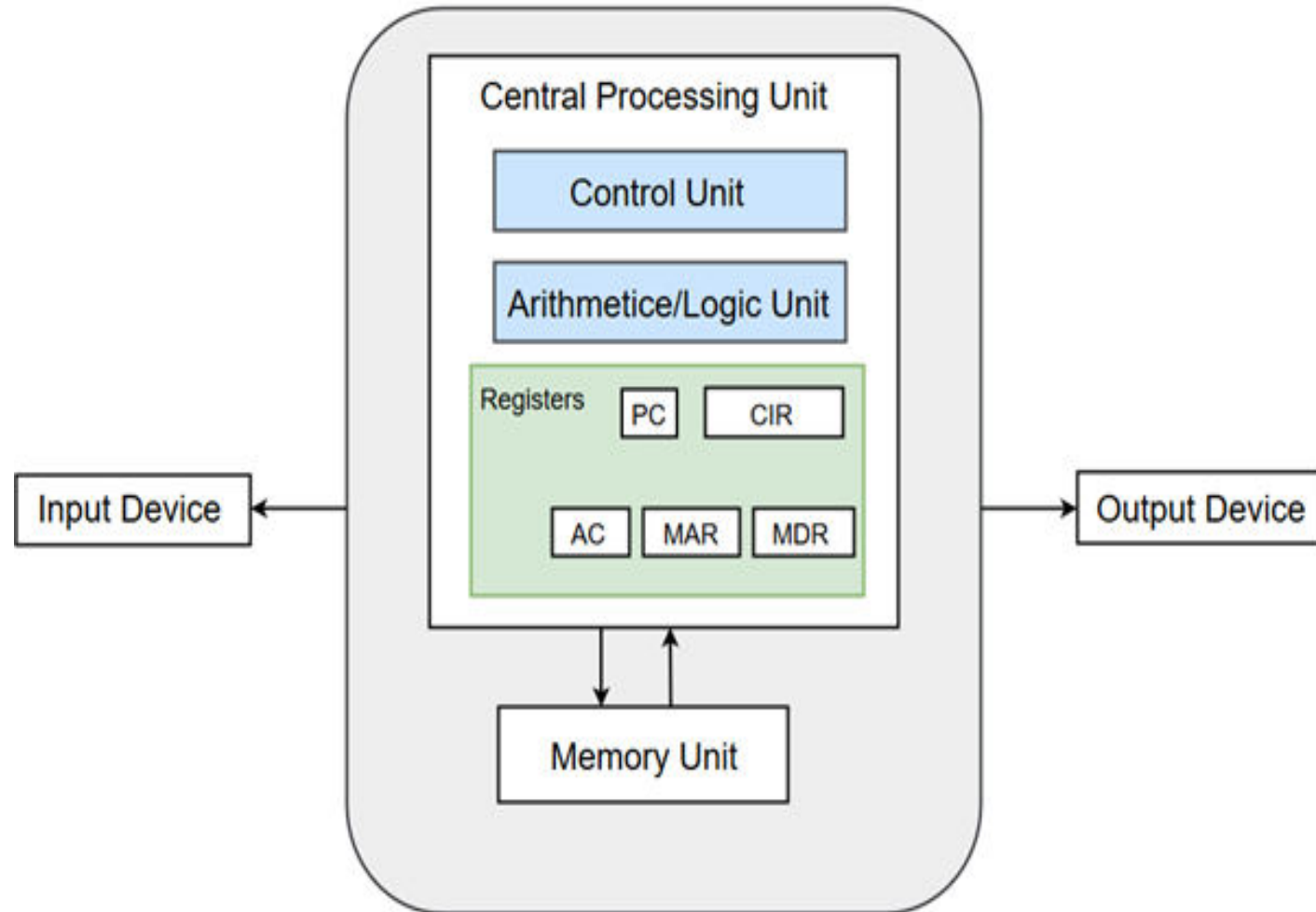


Von Neumann Model

- Computers built on the Turing model store data in their memory whereas John von Neumann proposed that, since program and data are logically the same, programs should also be stored in the memory of a computer.
- Von Neumann architecture is based on the **stored-program computer** concept, where instruction data and program data are stored in the same memory.
- It consist of a **Control Unit, Arithmetic, and Logical Memory Unit (ALU), Registers and Inputs/Outputs.**



Von Neumann Basic Structure



Sequential execution of instructions

- A program in the Von Neumann model is made of a finite number of instructions.
- Program and data are stored as binary patterns in memory—a sequence of 0s and 1s.
- The control unit fetches one instruction from memory, decodes it, then executes it.
- In other words, the instructions are executed one after another.
- Sequential execution of a program was the initial requirement of a computer based on the Von Neumann model.



What is Programming

- Program – a very specific set of instructions (or command lines) that making a computer do what you want it to do
- Types of Program-
 - System program
 - Application program
- Programming – the process of creating a program



Types of Programming Languages

- A programming language defines a set of instructions that are compiled together to perform a specific task by the CPU.
- These languages vary in the level of abstraction they provide from the hardware.
- Some programming languages provide less or no abstraction while some provide higher abstraction.
- Based on the levels of abstraction, they can be classified into two basic categories:
 1. Low-level language
 2. High-level language



Low Level Languages

- The low-level language is a programming language that provides no abstraction from the hardware, and it is represented in 0 or 1 forms, which are the machine instructions.
- The languages that come under this category are :
 - **Machine level language**
 - **Assembly language**



Computational Model: Revisited

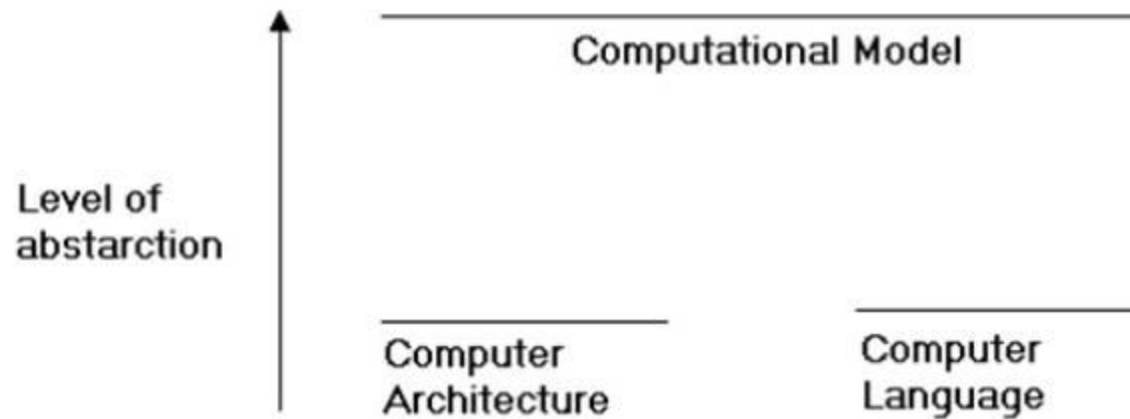


Figure 1.1

Machine Level Languages

- The machine-level language is a language that consists of a set of instructions that are in the binary form 0 or 1.
- Creating a program in a machine-level language is a very difficult task as it is not easy for the programmers to write the program in machine instructions.
- A machine-level language is not portable as each computer has its machine instructions, so if we write a program in one computer will no longer be valid in another computer.
- The different processor architectures use different machine codes.



Assembly Languages

- The assembly language contains some human-readable commands such as
 - **MOV, ADD, SUB etc.**
- It is easier to write and understand.
- As we know that computers can only understand the machine-level instructions, so we require a translator that converts the assembly code into machine code.
- The translator used for translating the code is known as an **assembler**.
- The assembly language code is not portable because the data is stored in computer registers, and the computer has to know the different sets of registers.



Difference between Machine-Level language and Assembly language

Machine-level language	Assembly language
The machine-level language comes at the lowest level in the hierarchy, so it has zero abstraction level from the hardware.	The assembly language comes above the machine language means that it has less abstraction level from the hardware.
It cannot be easily understood by humans.	It is easy to read, write, and maintain.
The machine-level language is written in binary digits, i.e., 0 and 1.	The assembly language is written in simple English language, so it is easily understandable by the users.
It does not require any translator as the machine code is directly executed by the computer.	In assembly language, the assembler is used to convert the assembly code into machine code.
It is a first-generation programming language.	It is a second-generation programming language.

High Level Language

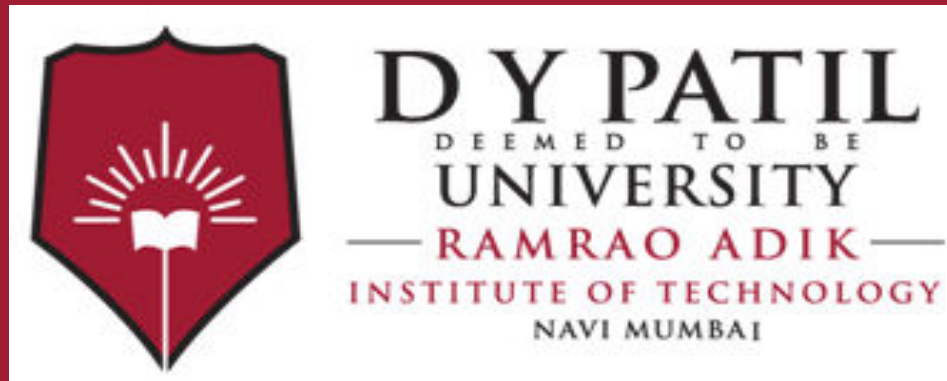
- The high-level language is a programming language that allows a programmer to write the programs which are independent of a particular type of computer.
- The high-level languages are considered as high-level because they are closer to human languages than machine-level languages.
- When writing a program in a high-level language, then the whole attention needs to be paid to the logic of the problem.
- A **compiler** is required to translate a high-level language into a low-level language.
- Examples : C, C++, Pascal



Difference between Low-Level language and High-Level language

Low-level language	High-level language
It is a machine-friendly language, i.e., the computer understands the machine language, which is represented in 0 or 1.	It is a user-friendly language as this language is written in simple English words, which can be easily understood by humans.
The low-level language takes more time to execute.	It executes at a faster pace.
It requires the assembler to convert the assembly code into machine code.	It requires the compiler to convert the high-level language instructions into machine code.
The machine code cannot run on all machines, so it is not a portable language.	The high-level code can run all the platforms, so it is a portable language.
It is memory efficient.	It is less memory efficient.
Debugging and maintenance are not easier in a low-level language.	Debugging and maintenance are easier in a high-level language.





Thank You

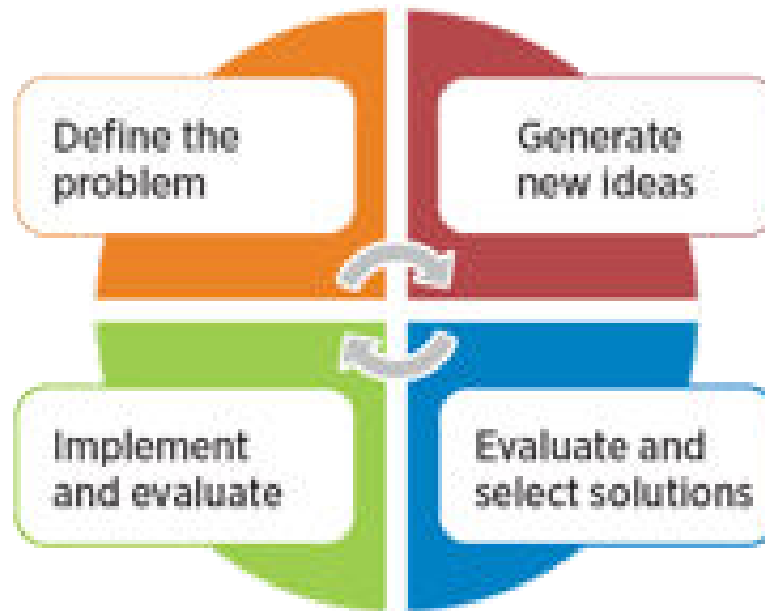
Lecture No: 2

Problem Solving and Algorithm



Problem Solving

Problem solving is the act of defining a **problem**; determining the cause of the **problem**; identifying, prioritizing, and selecting alternatives for a solution; and implementing a solution.



Steps in program development/ Problem Solving

1. Define the problem
2. Outline the solution
3. Develop the outline into an algorithm
4. Test the algorithm for correctness
5. Code the algorithm into a specific programming language
6. Run the program on the computer
7. Document and maintain the program



Define the Problem

- Divide the problem into three components (called IPO):
 - **Inputs** – what do you have?
 - **Outputs** – what do you want to have?
 - **Processing**
 - how do you go from inputs to outputs?



Outline the Solution

- The major processing steps involved
- The major subtasks (if any)
- The major control structures (e.g. repetition loops)
- The major variables and record structures
- The mainline logic



Develop the Outline into an Algorithm

- Algorithm is a set of precise steps that describe exactly the tasks to be performed, and the order in which they are to be carried out.
- Test data needs to be walked through each step in the algorithm, to check that the instructions described in the algorithm will actually do what they are supposed to



Steps in Problem Solving (Cont...)

- Write the programs in a specific programming language
- Execute the program and get the desired output.
- Prepare the documentation.



Representation of solution

- Pseudocode
- Algorithm
- Flowchart



Pseudocode

- The term pseudocode means writing steps required to perform a specific task, in any language that is spoken across the globe.
- It is written in informal language.
- Pseudocode cannot be compiled nor executed, and there are no real formatting or syntax rules.



Algorithm

- An algorithm is a sequence of finite logical steps required to perform a specific task.
- Algorithm is a set of precise steps that describe exactly the tasks to be performed, and the order in which they are to be carried out.
- It follows certain syntax.



Characteristics of Algorithms

1. **Finiteness:** An algorithm must terminate in a finite number of steps.
2. **Definiteness:** Each step of the algorithm must be precisely and unambiguously stated.
3. **Effectiveness:** Each step must be effective, in the sense that it should be primitive easily convert into program statement and can be performed exactly in a finite amount of time.
4. **Generality:** The algorithm must be complete in itself so that it can be used to solve problems of a specific type for any input data.
5. **Input/output:** Each algorithm must take zero, one or more quantities as input data produce one or more output values.



Example 1:

- Calculate the circumference and area of Circle
- Problem definition :IPO
 - **Input:** radius
 - **Process:** calculate circumference and area as
$$\text{circumference} = 2 * \pi * \text{radius}$$
$$\text{area} = \pi * \text{radius}^2$$
 - **Output:** circumference, area



Example 1:Pseudocode

Begin program

get radius, PI

circumference = $2 * PI * \text{radius}$

area = $PI * \text{radius} * \text{radius}$

display circumference, area

End program



Example 1: Algorithm

Step 1: Start

Step 2: Take input for radius

Step 3: $PI=3.14$

Step 4: $area=PI * radius * radius$

Step 5: $circumference=2 * PI * radius$

Step 6: Display radius, area and circumference

Step 7: Stop



Example 2 :

Q. Define problem “**To display addition of two numbers**”.

Problem definition :IPO

Input: num1, num2

Process: calculate sum as $\text{sum} = \text{num1} + \text{num2}$

Output: sum



Example 2 :

Q. Write an algorithm “**To display addition of two numbers**”.

Step 1 : Start.

Step 2 : Take input for two numbers num1 and num2

Step 3 : $\text{sum} = \text{num1} + \text{num2}$

Step 4 : Display sum.

Step 5 : Stop.



Example 3 :

Q. Define problem “**to find the average of three numbers**”.

Problem definition :IPO

Input: a, b, c

Process: calculate sum as $\text{sum} = a + b + c$
and average as $\text{avg} = \text{sum} / 3$

Output: avg.

Example 3 :

Q. Write an algorithm “**to find the average of three numbers**”.

Step 1 : Start.

Step 2 : Take input for the numbers a, b, c.

Step 3 : Compute $\text{sum} = a + b + c$.

Step 4 : Compute $\text{avg} = \text{sum} / 3$.

Step 5 : print value of avg.

Step 6 : Stop



Example 4 :

Q. Define problem “**to calculate the simple interest using the formula. Simple interest = $P * N * R / 100$.**”

Problem definition :IPO

Input: Principal P , Number of years N and Rate of interest R

Process: calculate Simple Interest as
 $SI = P * N * R / 100$

Output: Simple Interest SI



Example 4 :

Q. Write an algorithm “**to calculate the simple interest using the formula. Simple interest = $P * N * R / 100$.**”

Step 1 : Start.

Step 2 : Take input for P, N and R.

Step 3 : Calculate simple interest as $SI = P * N * R / 100$

Step 4 : Print SI.

Step 5 : Stop.



Example 5 :

Q. Define Problem “**to calculate the perimeter and area of rectangle. Given its length and width.**”

Problem definition :IPO

Input: Length and width

Process: calculate perimeter as

$\text{perimeter} = \text{length} * \text{width}$

calculate area as

$\text{area} = 2 * (\text{length} + \text{width})$

Output: Perimeter and area



Example 5 :

Q. Write an algorithm “**to calculate the perimeter and area of rectangle. Given its length and width.**”

Step 1 : Start.

Step 2 : Take input for length of the rectangle.

Step 3 : Take input for width of the rectangle.

Step 4 : Calculate perimeter of the rectangle using the formula $\text{perimeter} = 2 * (\text{length} + \text{width})$

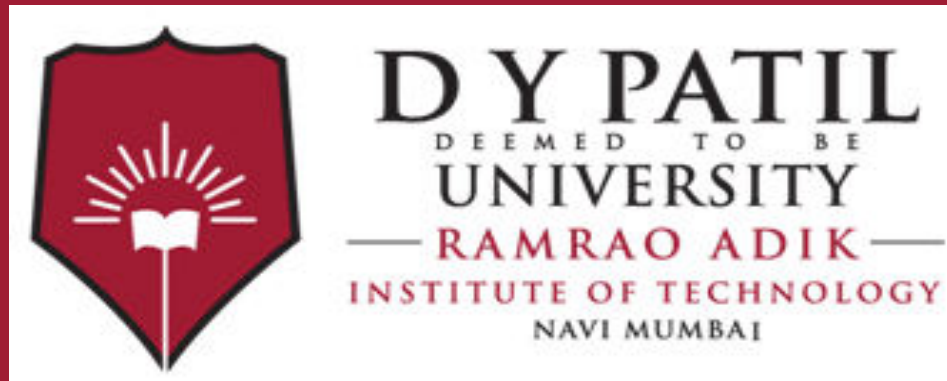
Step 5 : Calculate area of the rectangle using the formula $\text{area} = \text{length} * \text{width}$.

Step 6 : Print perimeter.

Step 7 : Print area.

Step 8 : Stop.





Thank You

Lecture No: 3

Flowchart



Example 6 :

Q. Write an algorithm “**to swap two numbers**”.

Step 1 : Start.

Step 2 : Read two numbers num1 and num2.

Step 3 : Declare temporary variable temp.

Step 4 : Interchange the data

temp = num1

num1 = num2

num2 = temp

Step 6 : Print the value of num1 and num2.

Step 7 : Stop.



Example 7 :

Q. Write an algorithm “**to find the largest of two numbers X, Y.**”

Step 1 : Start.

Step 2 : Read two numbers X and Y.

Step 3 : if $X > Y$

Print value of X as the largest number and go to step 5.

Step 4 : if $X < Y$

Print value of Y as the largest number and go to step 5.

Step 5 : Stop.

Example 8 :

Q. Write an algorithm “**to find the given number is even or odd.**”

Step 1 : Start.

Step 2 : Read number say Num.

Step 3 : if $\text{Num mod } 2 = 0$

Print “Number entered is even number.” and go to

Step 4 : if $\text{Num mod } 2 \neq 0$

Print “Number entered is odd number.” and go to

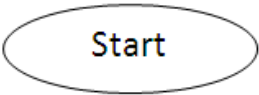
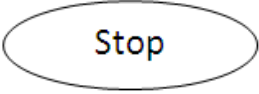
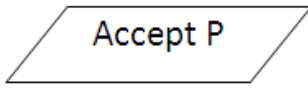
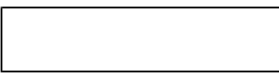
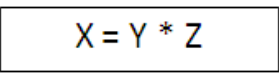
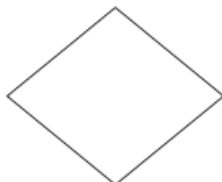
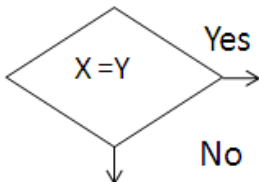
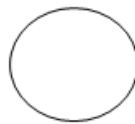
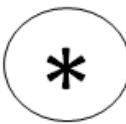
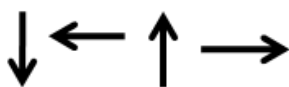
Step 5 : Stop.

Flowchart

- A flowchart is a **pictorial representation** of a sequence of steps to be performed for solving a given problem
- Or A flowchart is **visual or graphical representation of an algorithm** that illustrates the sequence of operations to be performed to get a solution.
- The flowcharts helps to analyze the problem and plan its solution in a systematic and orderly manner.
- A flowchart when translated in to a proper computer language, results in a complete program.



Symbols used in Flowchart

Symbol	Name	Purpose	Example	Description
	Oval	Terminal		Start execution sequence
				Stop execution sequence
	Parallelogram	Input / Output		Accept value of variable P
				Print value of variable P
	Rectangle	Process		Multiply value of Y and Z, then store it in X
	Diamond	Decision		If $X = Y$, then follow path indicated by Yes; otherwise, follow the path indicated by No
	Circle	Connector		An entry at the connecting point represented by * in flowchart
	Arrow	Flow line		Guides the path to follow in next step

Guidelines for preparing Flowcharts

- The flowchart should be clear, neat and easy to follow.
- The flowchart must have logical start and finish.
- Only one flow line should come out from a process symbol.
- Only one flow line should enter a decision symbol. However two or three flow lines (one for each possible answer) may leave the decision symbol.
- In case of complex flowcharts connector symbols are used to reduce the number of flow lines.

Example 1

Step 1: Start

Step 2: Take input for radius

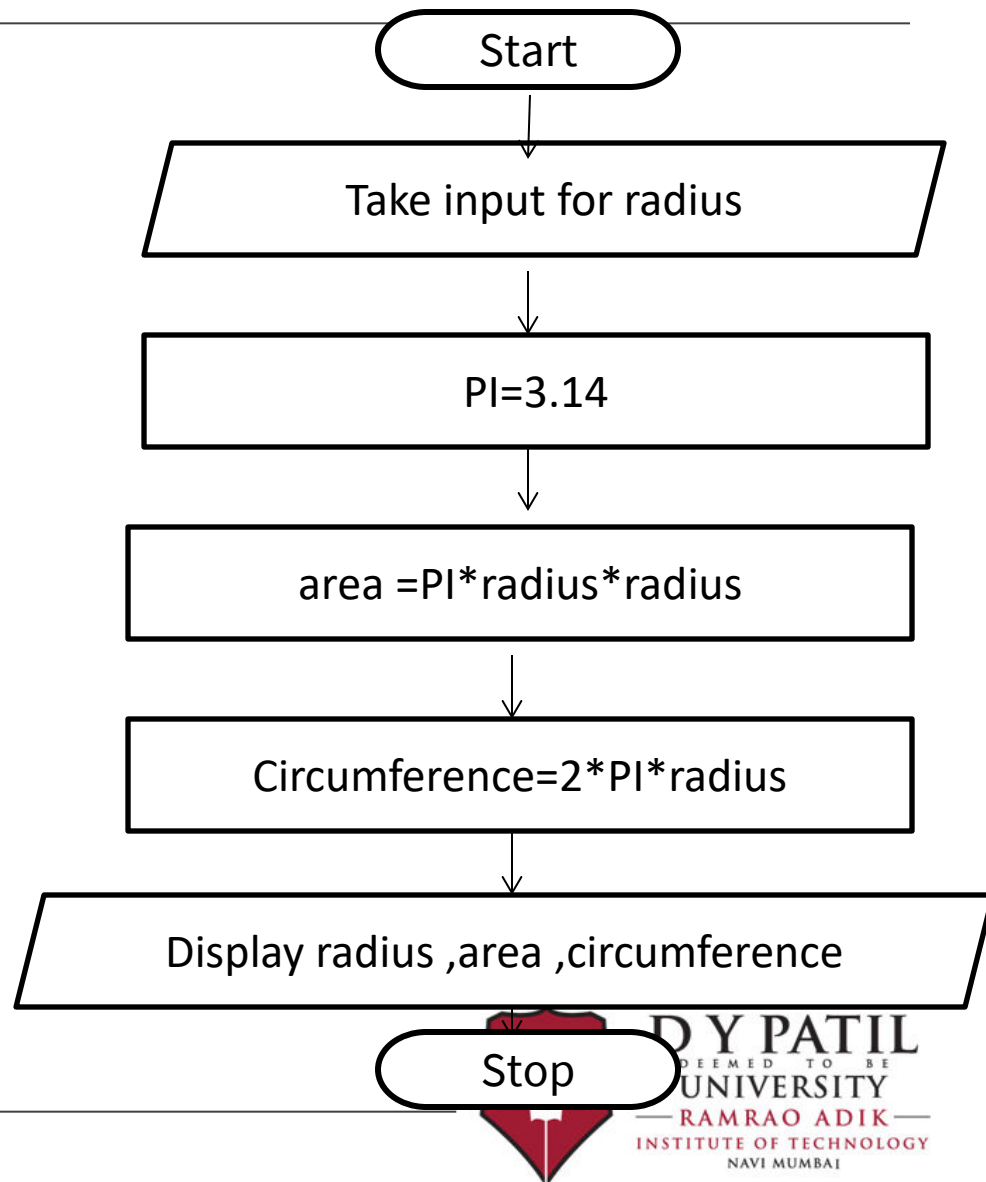
Step 3: $\text{PI}=3.14$

Step 4: $\text{area}=\text{PI} \times \text{radius} \times \text{radius}$

Step 5: $\text{circumference}=2 \times \text{PI} \times \text{radius}$

Step 6: Display radius, area and circumference

Step 7: Stop



Example 2 :

Q. Draw a flowchart “**to find the average of three numbers**”.

Step 1 : Start.

Step 2 : Read the numbers a, b, c.

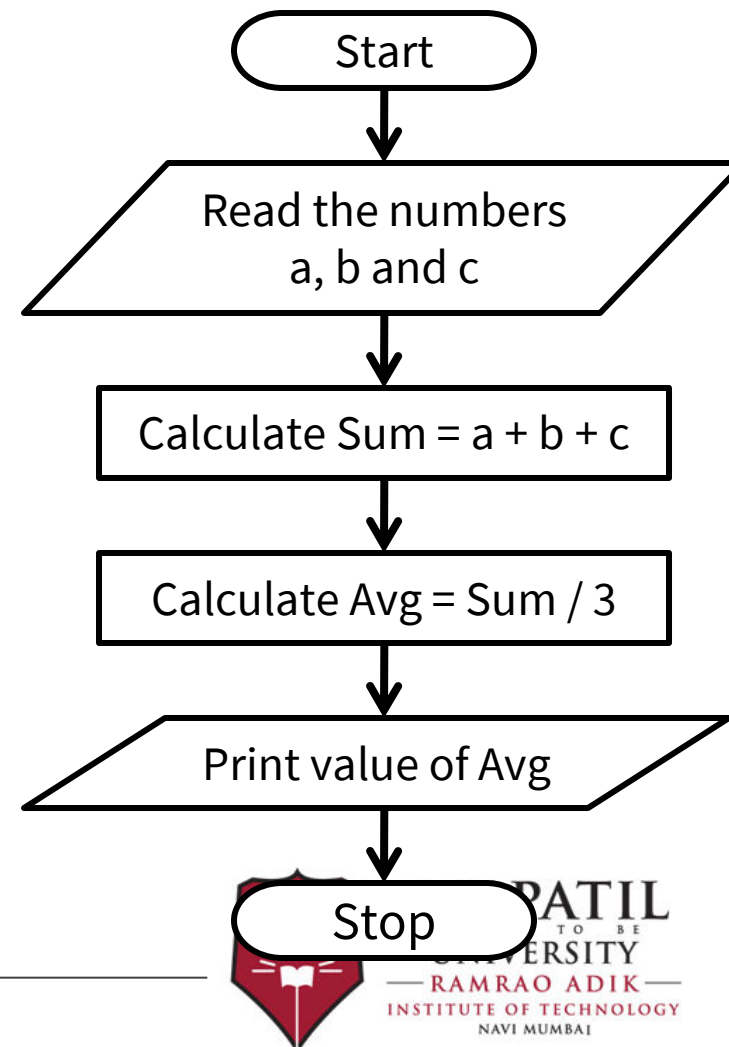
Step 3 : Compute the sum of a, b and c.

Step 4 : Divide the sum by 3.

Step 5 : Store the result in variable Avg.

Step 6 : Print the value of Avg.

Step 7 : Stop.



Example 3 :

Q. Draw a flowchart “**to calculate the simple interest using the formula. Simple interest = $P \cdot N \cdot R / 100$.**”

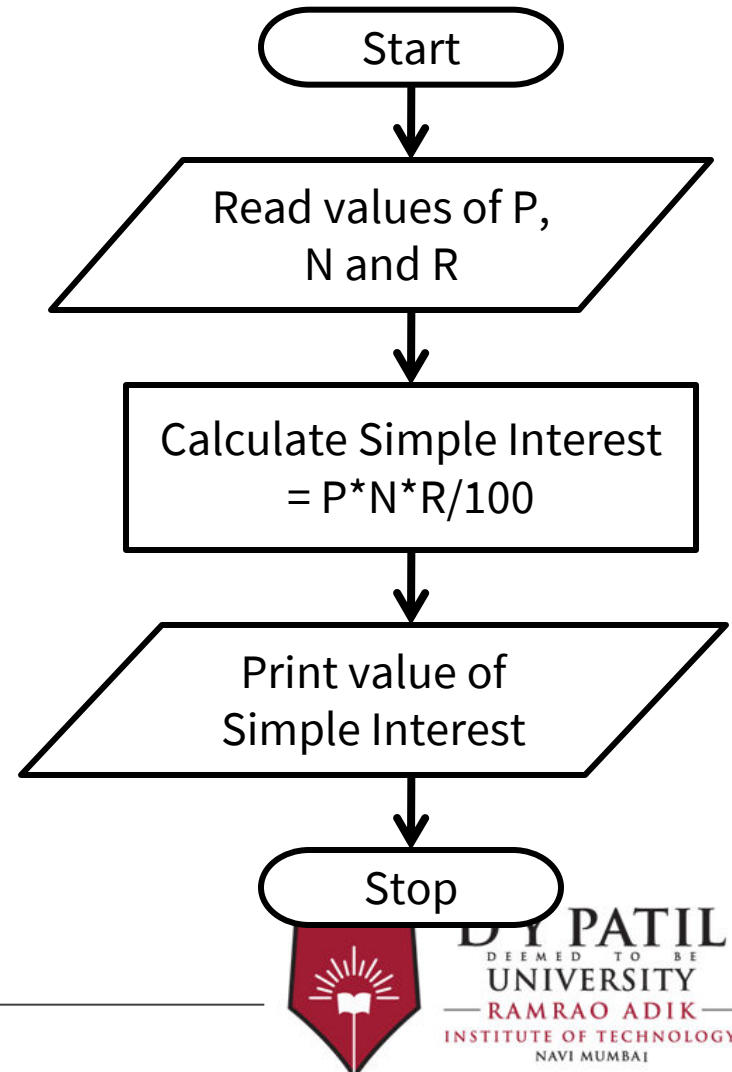
Step 1 : Start.

Step 2 : Read the three input quantities' P, N and R.

Step 3 : Calculate simple interest as
Simple interest = $P \cdot N \cdot R / 100$

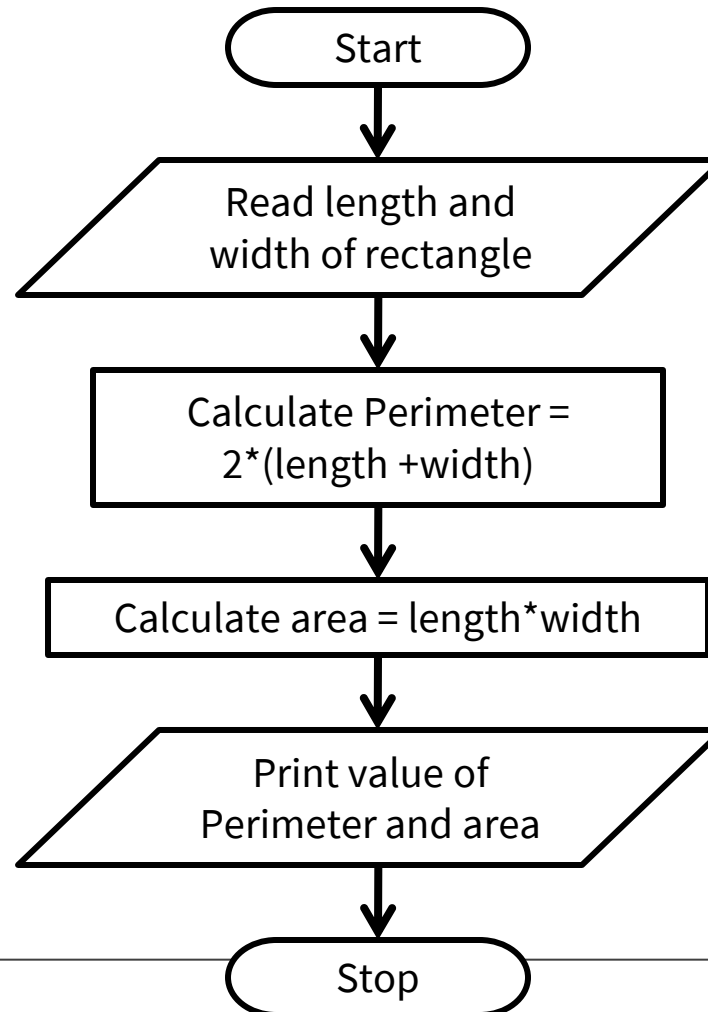
Step 4 : Print simple interest.

Step 5 : Stop.



Example 4 :

Q. Draw a flowchart “**to calculate the perimeter and area of rectangle. Given its length and width.**”



Benefits of a flowchart

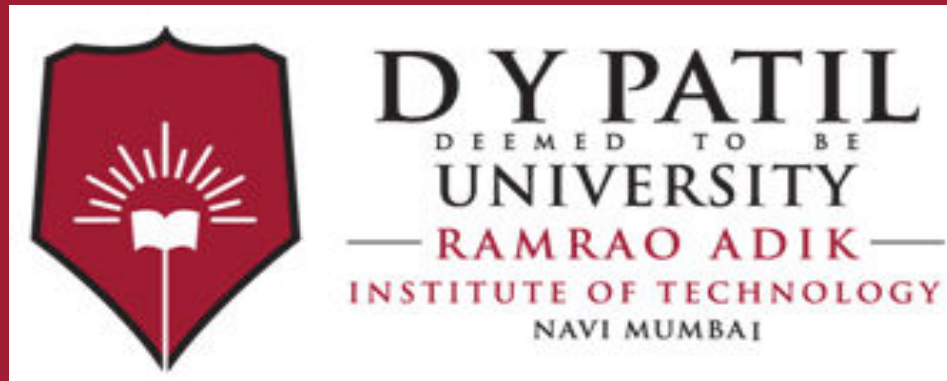
- It is easy to understand because of limited symbols used
- It provides convenient and easy approach to represent a solution to a problem
- It assists the user in reviewing and correcting the solution

Limitations of Flowcharts

- **Complex** : The major disadvantage in using flowcharts is that when a program is very large, the flowcharts may continue for many pages, making them hard to follow.
- **Costly** : If flowcharts are to be drawn for huge application the time and cost factor of program development may get out of proportion making it costly.
- **Difficult to Modify** : Due to its symbolic nature any changes or modification to a flowchart usually requires redrawing an entire logic again and redrawing a complex flowchart is not a simple task.
- **No Update** : Usually programs are updated regularly. However the corresponding update of flowcharts may not take place especially in the case of large programs. As a result the logic in the flowchart may not match with the actual program's logic.

Difference between Algorithm and Flowchart

Algorithm	Flowchart
A method of representing the step-by-step logical procedure for solving a problem	Flowchart is diagrammatic representation of an algorithm.
It contains step-by-step English descriptions, each step representing a particular operation leading to solution of problem	The flowchart employs a series of blocks and arrows, each of which represents a particular step in an algorithm
These are particularly useful for small problems	These are useful for detailed representations of small programs
For complex programs, algorithms prove to be adequate	For complex programs, Flowcharts prove to be inadequate



Thank You

Index -

Lecture 4 – Flowchart with Sequence and Decision	3
Lecture 5 – Flowchart with Repetition	19
Lecture 6 – Structure of C program	35



Lecture No: 4

Flowchart with Sequence and Decision



Three construct of Algorithm and flowchart

- The algorithm and flowchart can be classified to three types of *control structures*.

1. Sequence

2. Branching (Selection)

3. Loop (Repetition)



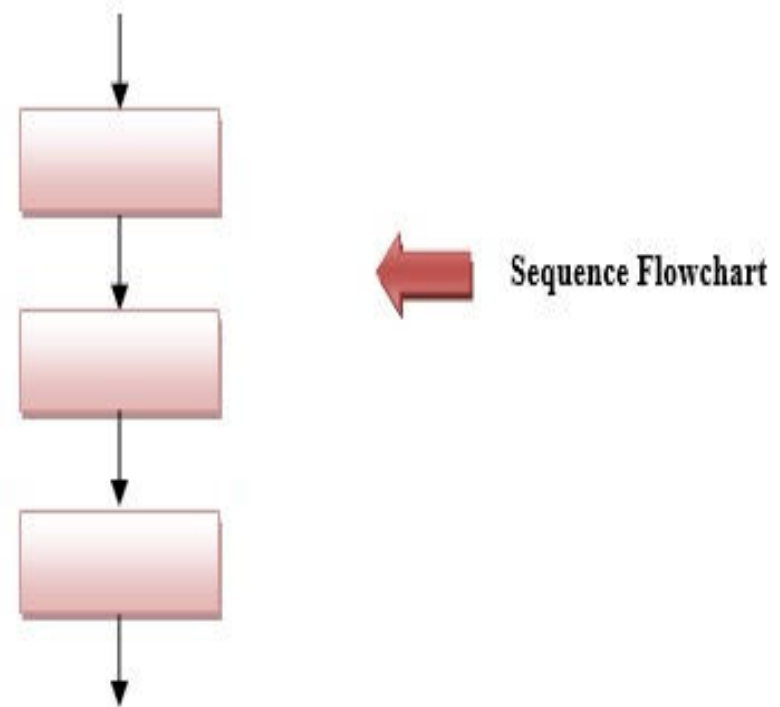
Flowchart : Sequence Construct

- The sequence is exemplified **by sequence of statements place one after the other** – the one above or before another gets executed first.
- In flowcharts, sequence of statements is usually contained in the rectangular process box.
- A sequence construct tells the processors which statement is to be executed next.
- In a sequence construct the **program flow lines simply move from one statement to the next.**



Flowchart : Sequence Construct (cont..)

- Sequence is the most basic form of the constructs.
- **It is simply performing one step after another.**
- Each step is followed in a specific sequence, hence the name.
- **Sequence can be thought of as “do this, then do this, then do this”.**



Example 4 :

Q. Write an algorithm and draw flowchart
“to swap two numbers”.

Step 1 : Start.

Step 2 : Read two numbers num1 and num2.

Step 3 : Declare temporary variable temp.

Step 4 : Interchange the data

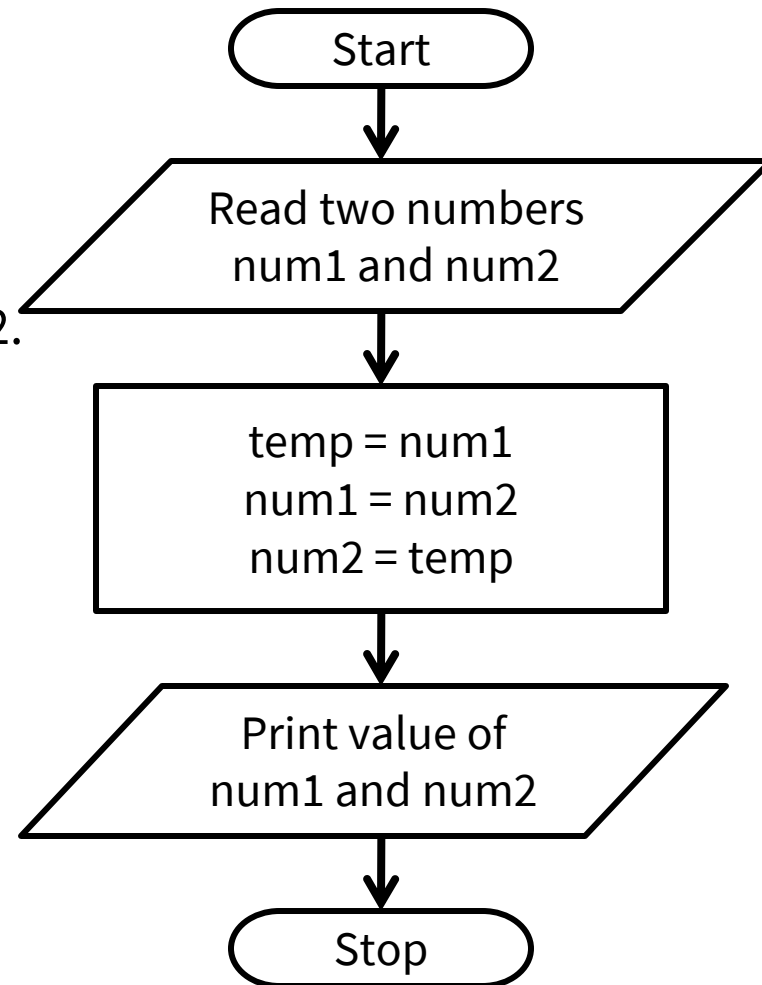
temp = num1

num1 = num2

num2 = temp

Step 5 : Print the value of num1 and num2.

Step 6 : Stop.



Example 5 :

Q. Write an algorithm and draw flowchart “**to convert temperature from Celsius to Fahrenheit**”.

Step 1 : Start.

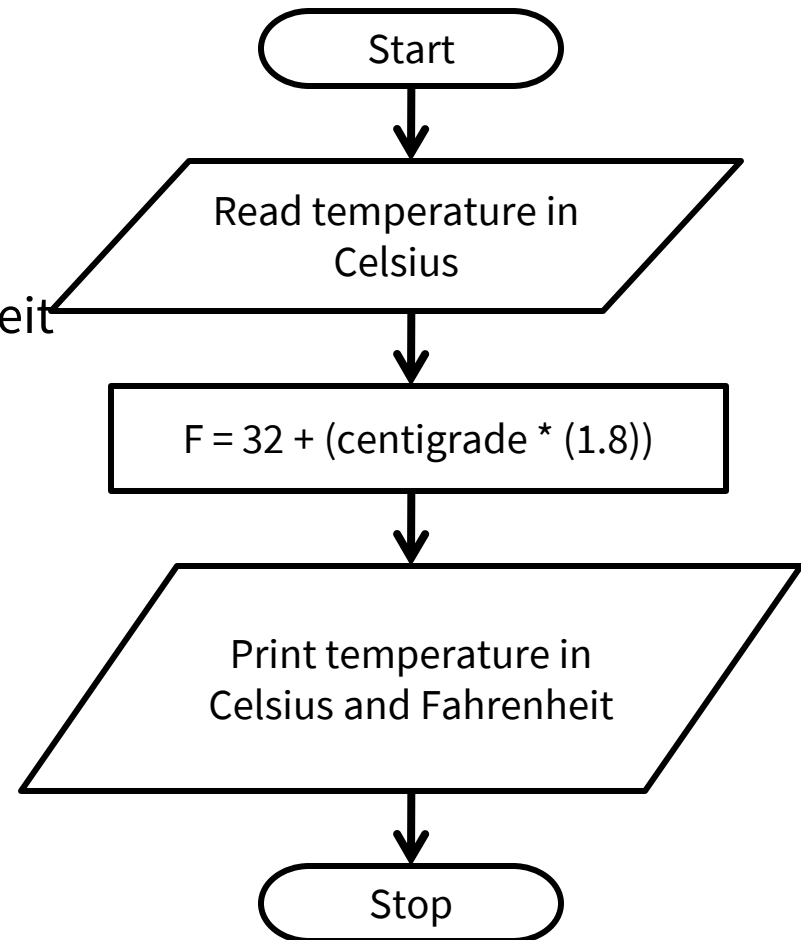
Step 2 : Read temperature in Celsius.

Step 3 : Calculate temperature in Fahrenheit

$$F = 32 + (\text{centigrade} * (1.8))$$

Step 4 : Display temperature in Celsius
and Fahrenheit.

Step 5 : Stop.



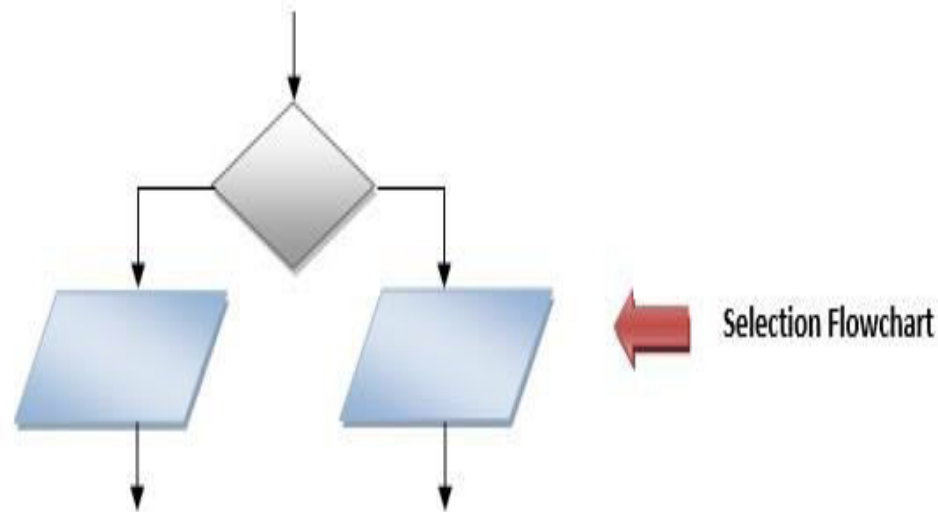
Flowchart : Decision Construct

- Flowchart with decision constructs refers to a binary decision based on some condition specified in the problem.
- If the condition is true, one of the two branches is explored; if the condition is false, the other alternative is taken.
- **This is usually represented by the 'if-then' construct in pseudo-codes and programs.**
- In flowcharts, this is represented by the diamond-shaped decision box.
- This structure is also known as the selection structure.



Flowchart : Decision Construct (cont..)

- **Selection logic, also known as decision logic, is used for making decisions.**
- **Selection logic is depicted as either an IF...THEN...ELSE or IF.....THEN structure.**
- It is used to make yes/no or true/false decisions logically.
- Selection can be thought of as “if something is true, take this action, otherwise take that action.



Example 1 :

Q. Write an algorithm “to print “YES” if number is greater than 10”

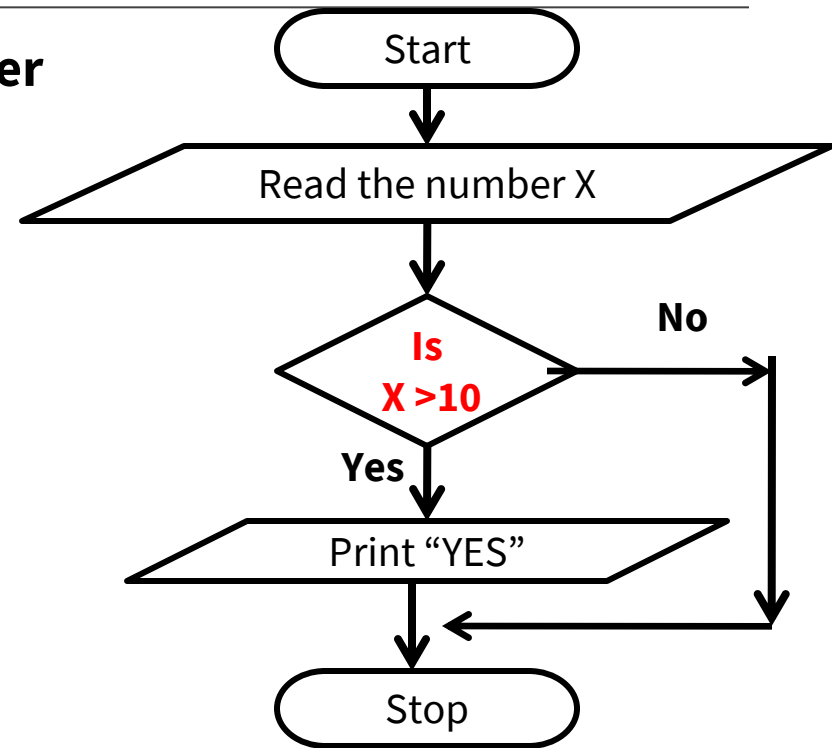
Step 1 : Start.

Step 2 : Read one number X .

Step 3 : if $X > 10$

Print as “YES”

Step 4: Stop.



Example 2 :

Q. Write an algorithm “to find the largest of two numbers X, Y.”

Step 1 : Start.

Step 2 : Read two numbers X and Y.

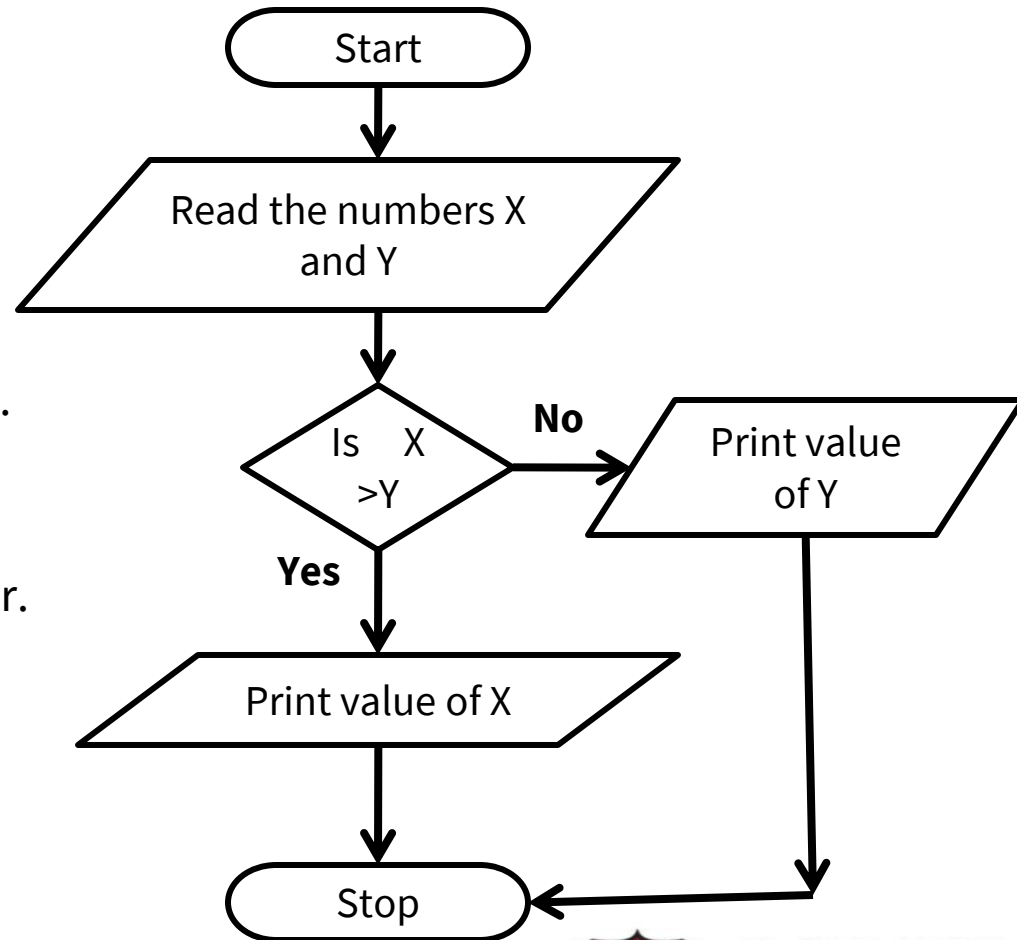
Step 3 : **if** $X > Y$

Print value of X as the largest number.

else

Print value of Y as the largest number.

Step 4 : Stop.



Example 3 :

Q. Write an algorithm “**to find the given number is even or odd.**”

Step 1 : Start.

Step 2 : Read number say Num.

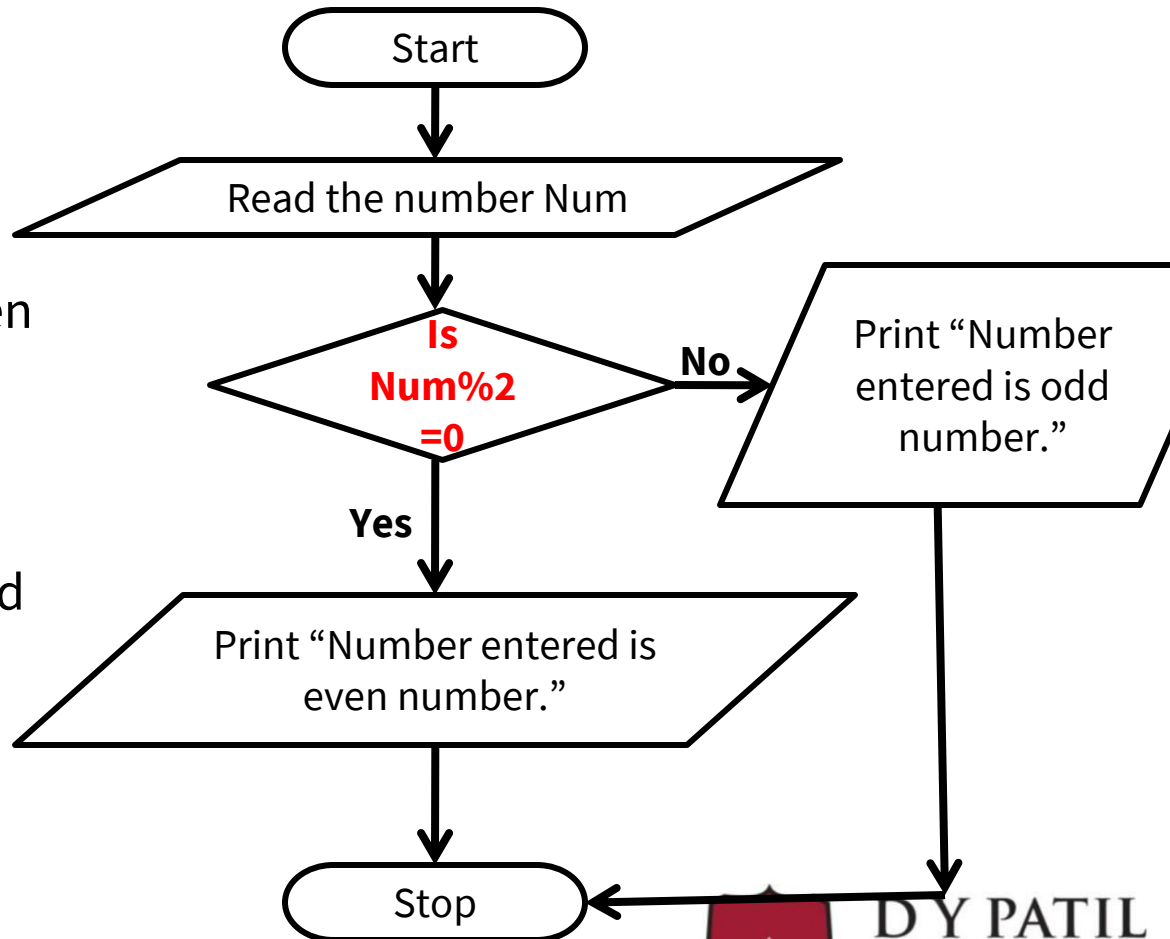
Step 3 : **if** $\text{Num} \bmod 2 = 0$

Print “Number entered is even number.”

else

Print “Number entered is odd number.”

Step 4 : Stop.



Example 4 :

Q. Write an algorithm “**to find out the biggest of the three unequal positive numbers.**”

Step 1: Start

Step 2: Declare variables a, b and c.

Step 3: Read variables a, b and c.

Step 4: If $a > b$

If $a > c$

Display a is the largest number.

Else

Display c is the largest number.

Else

If $b > c$

Display b is the largest number.

Else

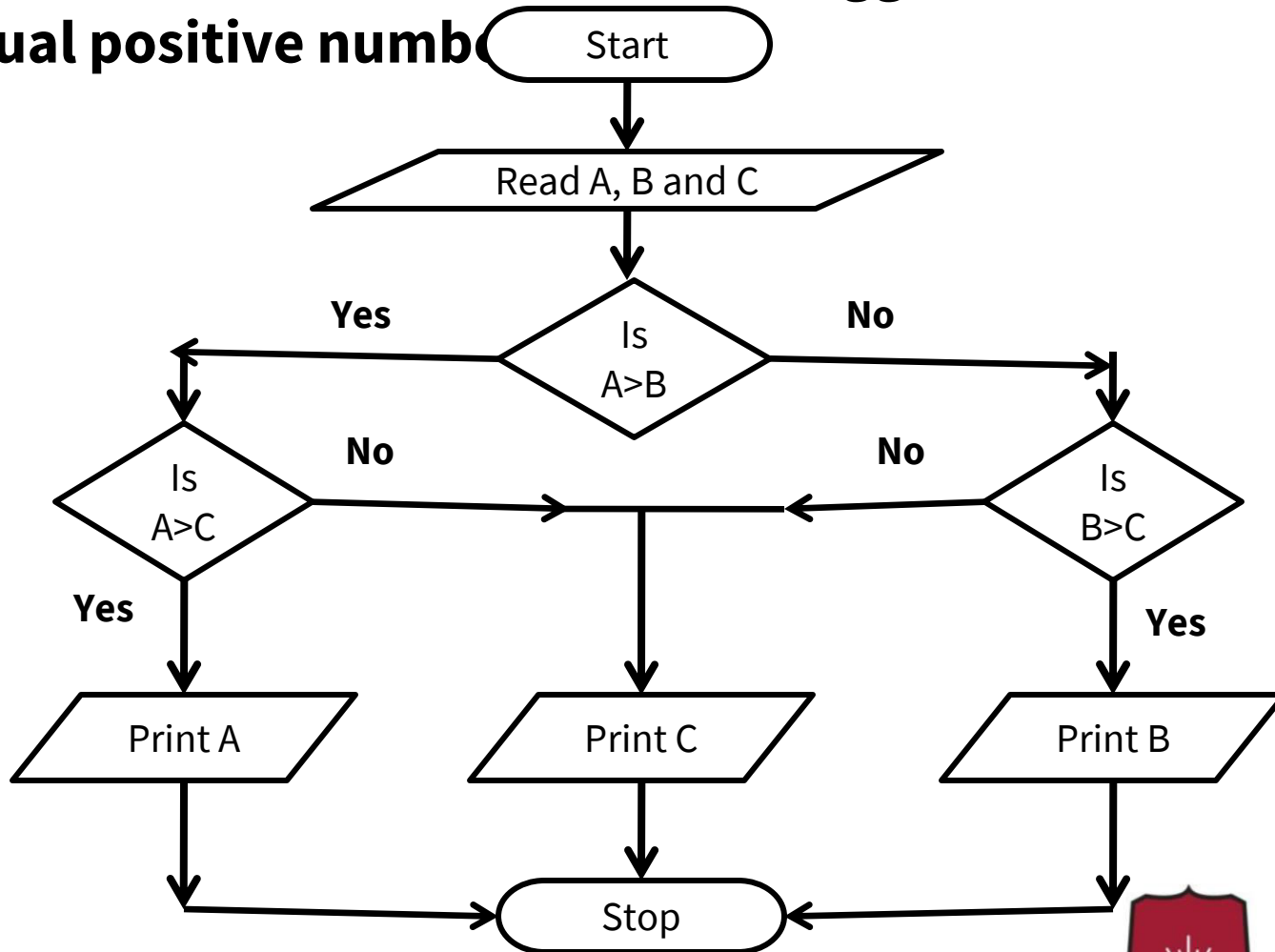
Display c is the greatest number.

Step 5: Stop.



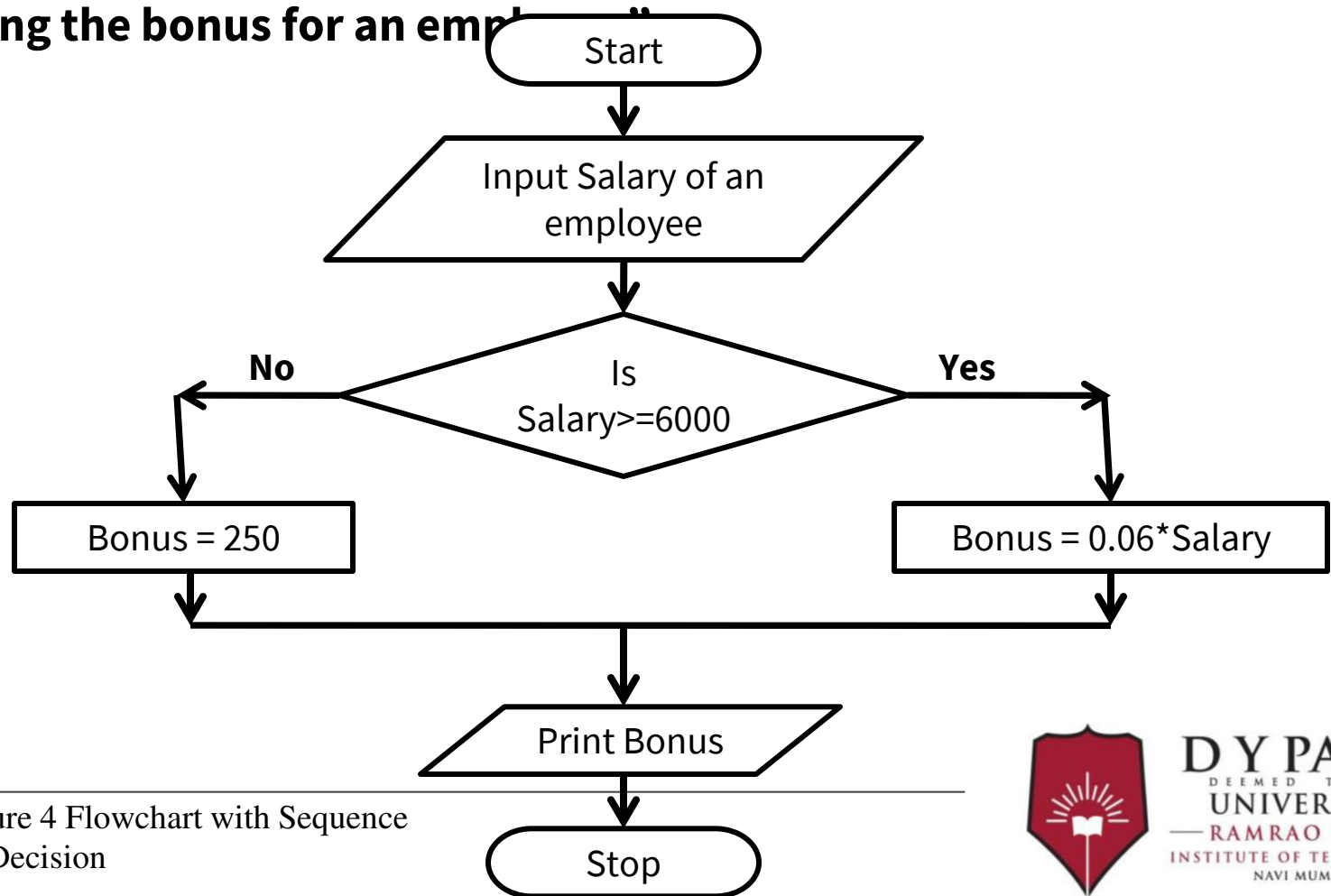
Example 5 :

Q. Draw a flowchart “to find out the biggest of the three unequal positive numbers”



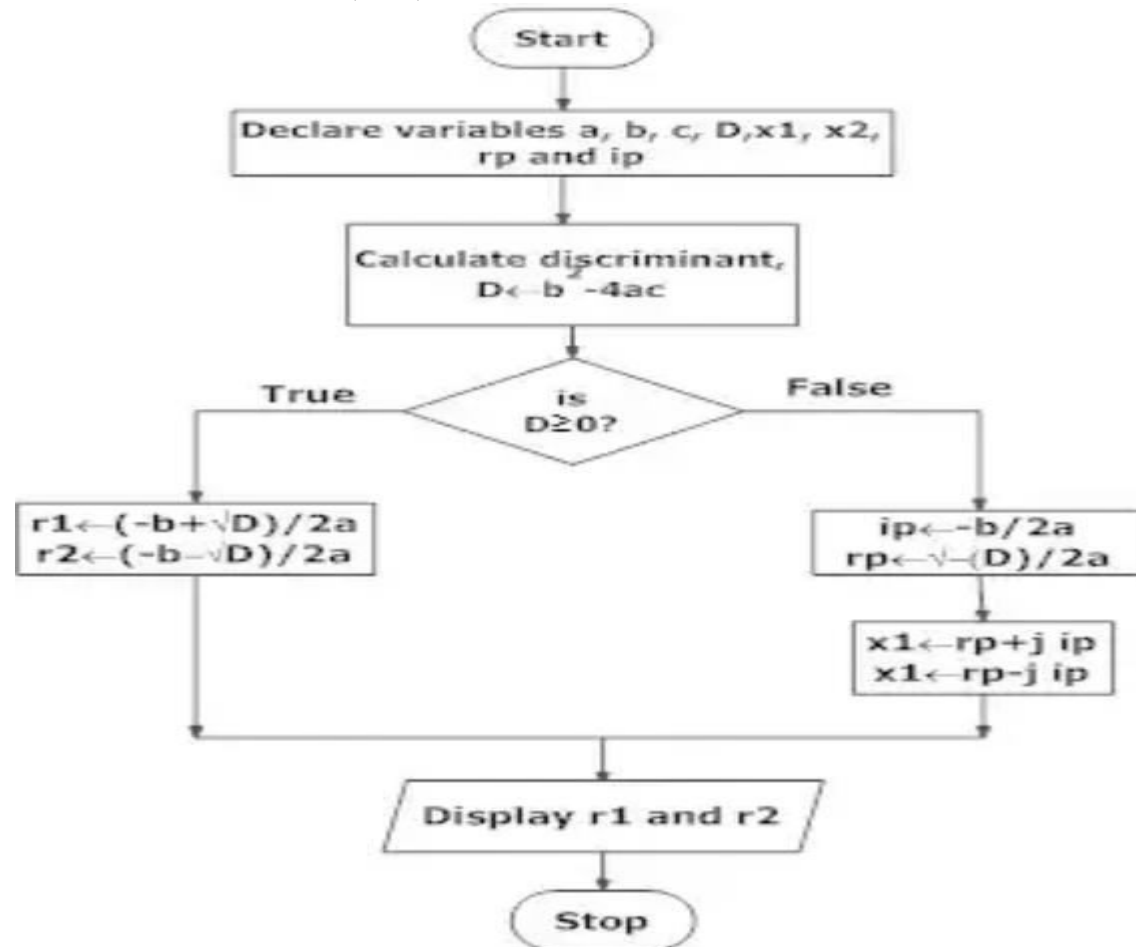
Example 6 :

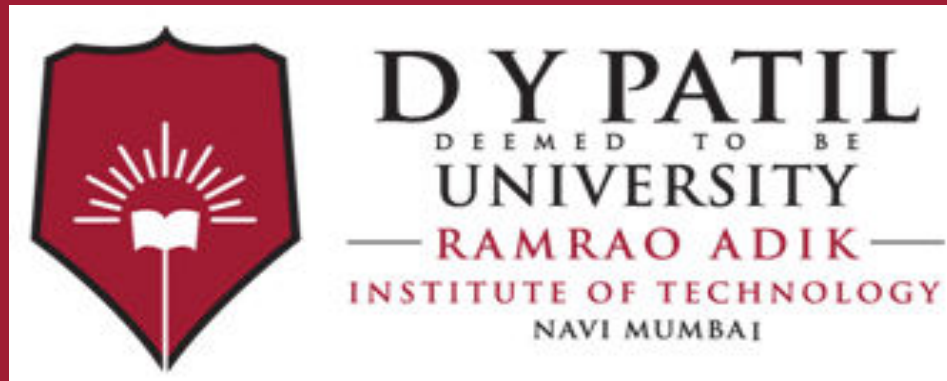
Q. Draw a flowchart “ABC company plans to give a 6% year-end bonus to each of its employees earning Rs 6,000 or more per month , and a fixed Rs 250 bonus to the remaining employees. Draw a flowchart for calculating the bonus for an employee”



Example 7 :

Q. Draw a flowchart “to find Roots of Quadratic equation $ax^2 + bx + c = 0$. The coefficients a, b, c are the input data ”





Thank You

Lecture No: 5

Flowchart with Repetition and Pseudo Code



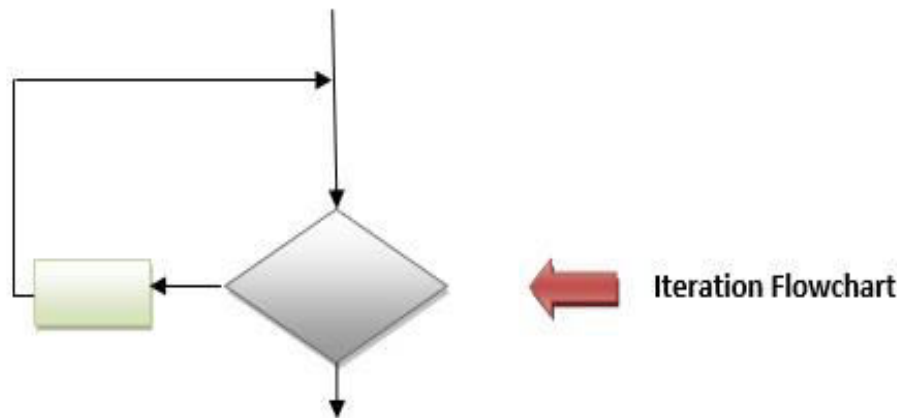
Repetition (Loop)

- The *loop* allows a statement or a sequence of statements to be repeatedly executed based on some loop condition.
- **A trip around the loop is known as iteration.**
- We must ensure that the condition for the termination of the looping must be satisfied after some finite number of iterations, otherwise it ends up as an infinite loop.
- **The loop is also known as the *repetition* structure.**
- It is represented by the '**while**', '**do-while**' and '**for**' control structures in most programming languages.



Repetition (Loop) (cont..)

- Iteration comes from the word “reiterate”, which means to repeat
- Iteration is a combination of decision and sequence and can repeat steps
- Iteration can be thought of as **“while something is true, do this, otherwise stop”**.
- **Usually the loop has two important parts:**
 1. An expression that is tested for a true/false.
 2. A statement or block that is repeated as long as the expression is true.



Example 1 :

Q. Write an algorithm and draw flowchart “**to find sum of 1 to N numbers.**”

Step 1 : Start.

Step 2 : Read number N.

Step 3 : Initialize counter, $i=1$ and $\text{Sum}=0$

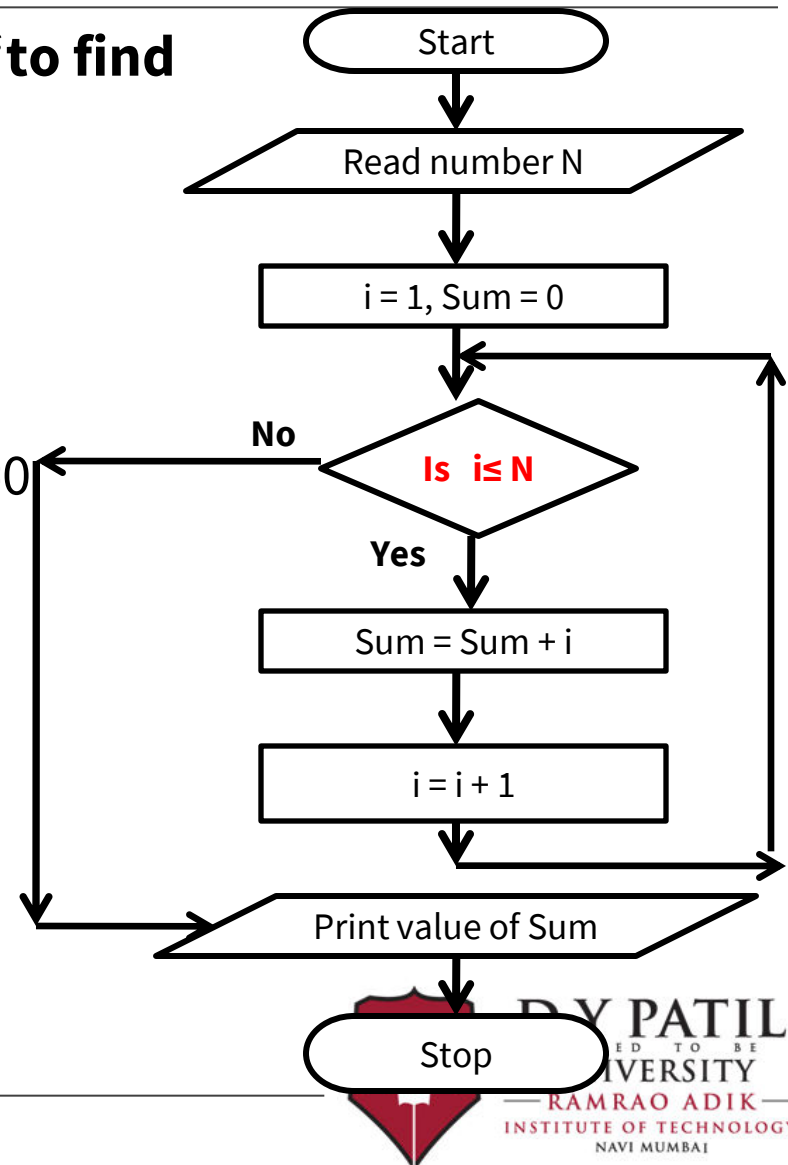
Step 4 : repeat until $i \leq N$

Calculate $\text{Sum} = \text{Sum} + i$

Increment counter, $i = i + 1$

Step 5 : Print value of Sum.

Step 6 : Stop.



Example 3 :

Q. Write an algorithm and draw flowchart “to find sum of square of 1 to N numbers i.e. $1^2 + 2^2 + \dots + N^2$ ”

Step 1 : Start.

Step 2 : Read number N.

Step 3 : Initialize counter, $i=1$ and $\text{Sum}=0$

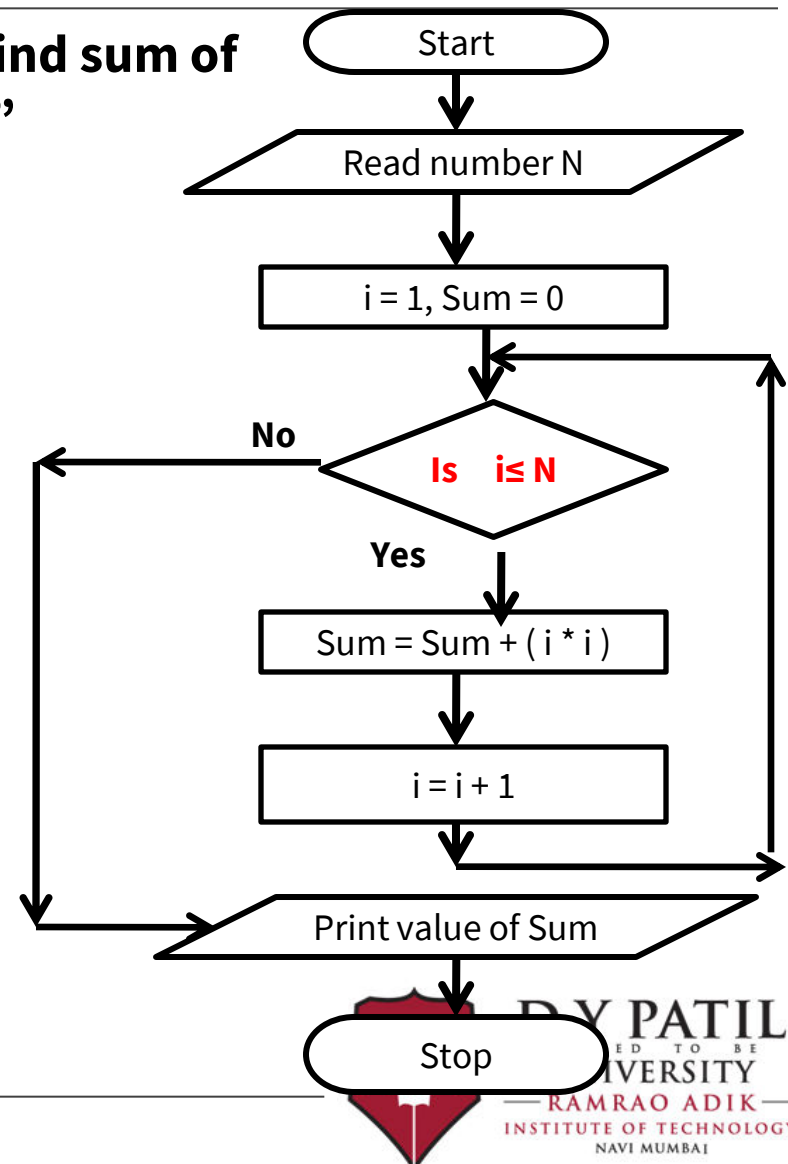
Step 4 : repeat until $i \leq N$

Calculate $\text{Sum} = \text{Sum} + (i * i)$

Increase counter, $i = i + 1$

Step 5 : Print value of Sum.

Step 6 : Stop.



Example 4 :

Q. Write an algorithm and draw flowchart “to display even numbers from 1 to 20 numbers.”

Step 1 : Start.

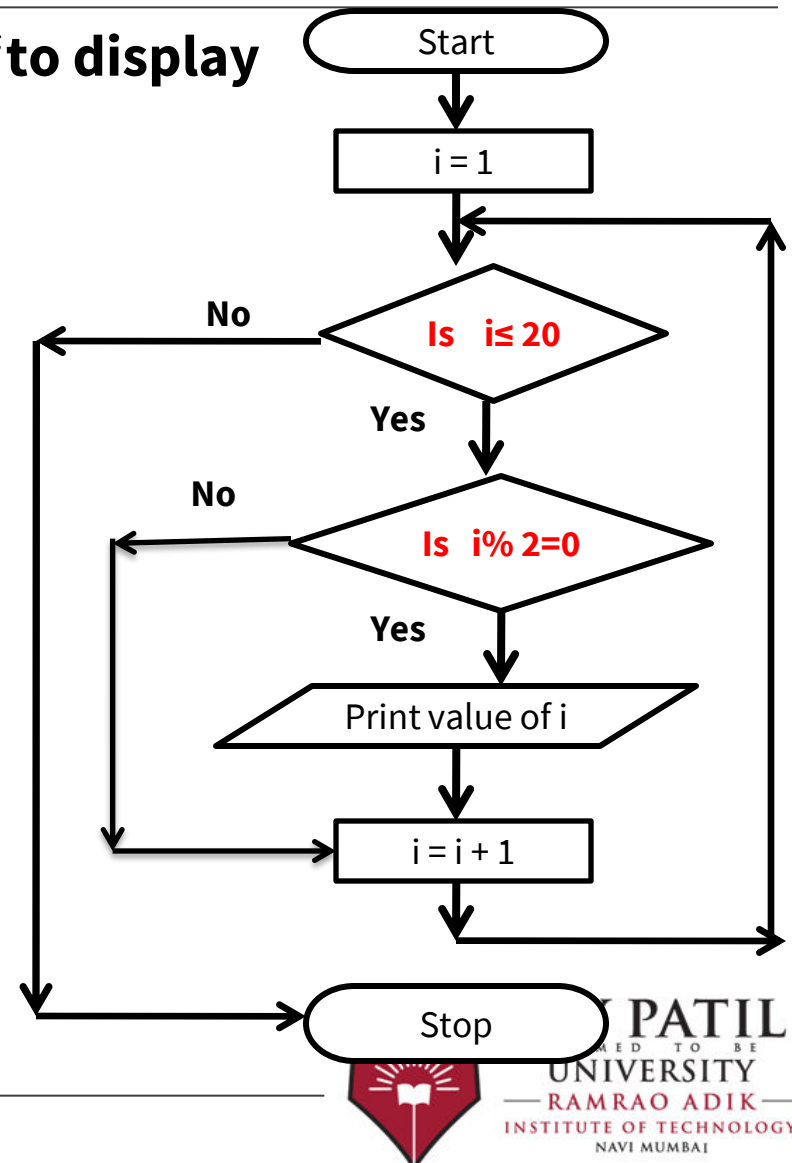
Step 2 : Initialize counter, $i = 1$

Step 3 : repeat until $i \leq 20$
if $i \% 2 = 0$

Print value of i

Increment counter, $i = i + 1$

Step 4 : Stop.



Example 5 :

Q. Write an algorithm and draw flowchart “**to display factorial of a number.**”

Step 1 : Start.

Step 2 : Read number N.

Step 3 : Initialize counter, $i=1$ and $f=1$

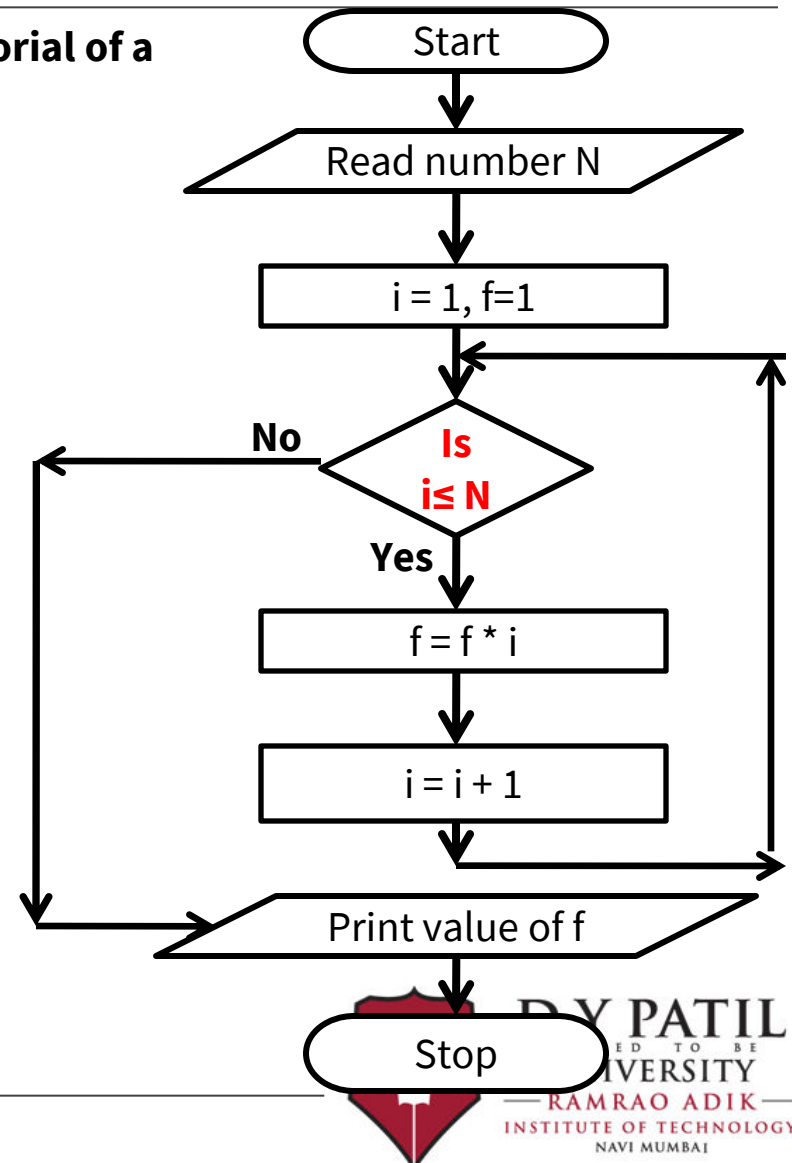
Step 4 : repeat until $i \leq N$

Calculate $f = f * i$

Increment counter, $i = i + 1$

Step 5 : Print value of f i.e. Factorial of N.

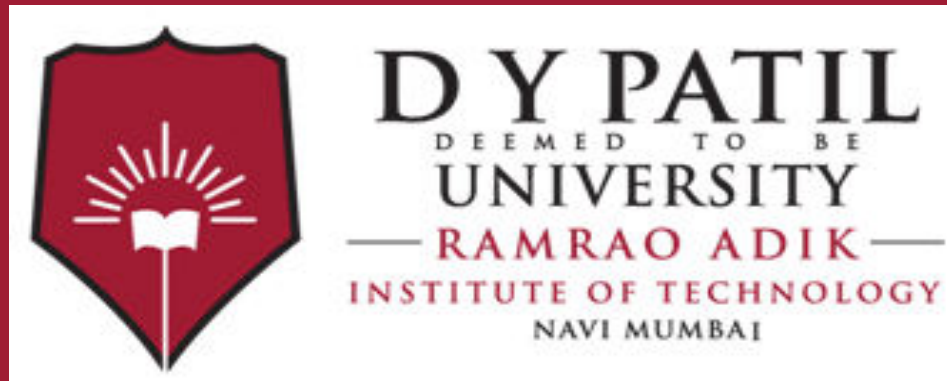
Step 6 : Stop.



Model Questions :

1. Define Algorithm.
2. What is Flowchart?
3. What is Pseudo code?
4. What are the symbols of Flowchart?
5. Write an algorithm and draw flowchart for calculating area and perimeter of Triangle.
6. What are the basic steps involved In problem solving?
7. Differentiate between Algorithm and Flowchart.
8. Write an algorithm and draw flowchart to find smallest of given three numbers.
9. Write an algorithm and draw flowchart to find odd numbers from M to N numbers.
10. Write an algorithm and draw flowchart to find cube of even numbers from 1 to N.





Thank You

Lecture No: 6

History of C Programming, Benefits of C, Structure of C program



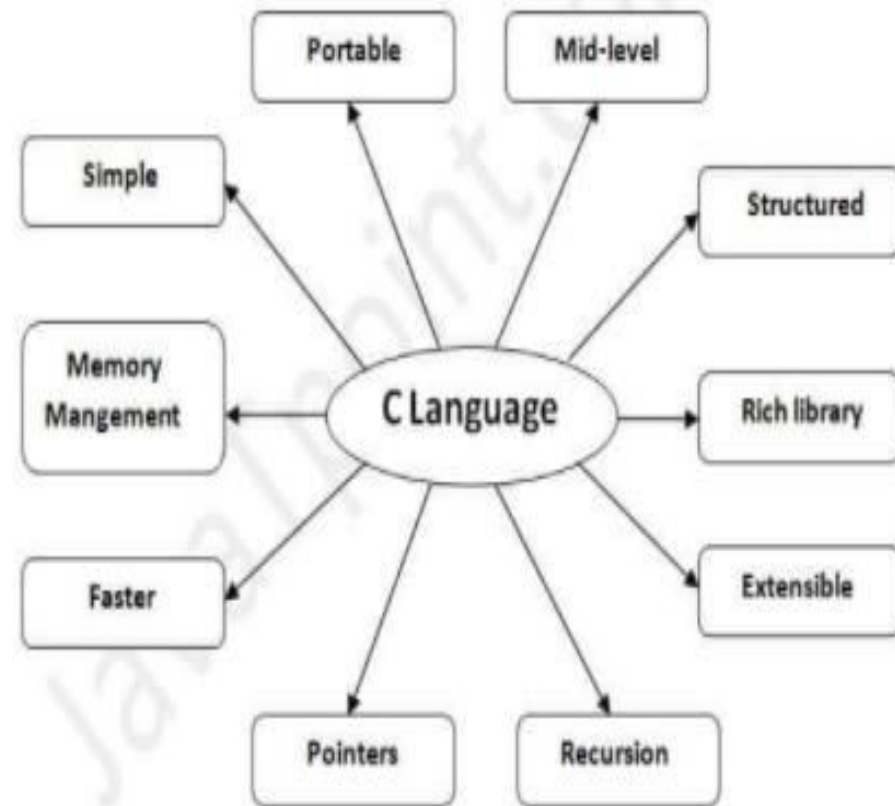
History of C Programming

Year	Language	Developed by	Drawbacks
1960	ALGOL	International Group	Too general and too abstract
1967	BCPL	Martin Richard	Less powerful and too specific
1970	B	Ken Thompson	Too specific
1972	Traditional C	Dennis Richie	
1989	ANSI C	ANSI Committee	
1990	ANSI/ISO C	ISO Committee	



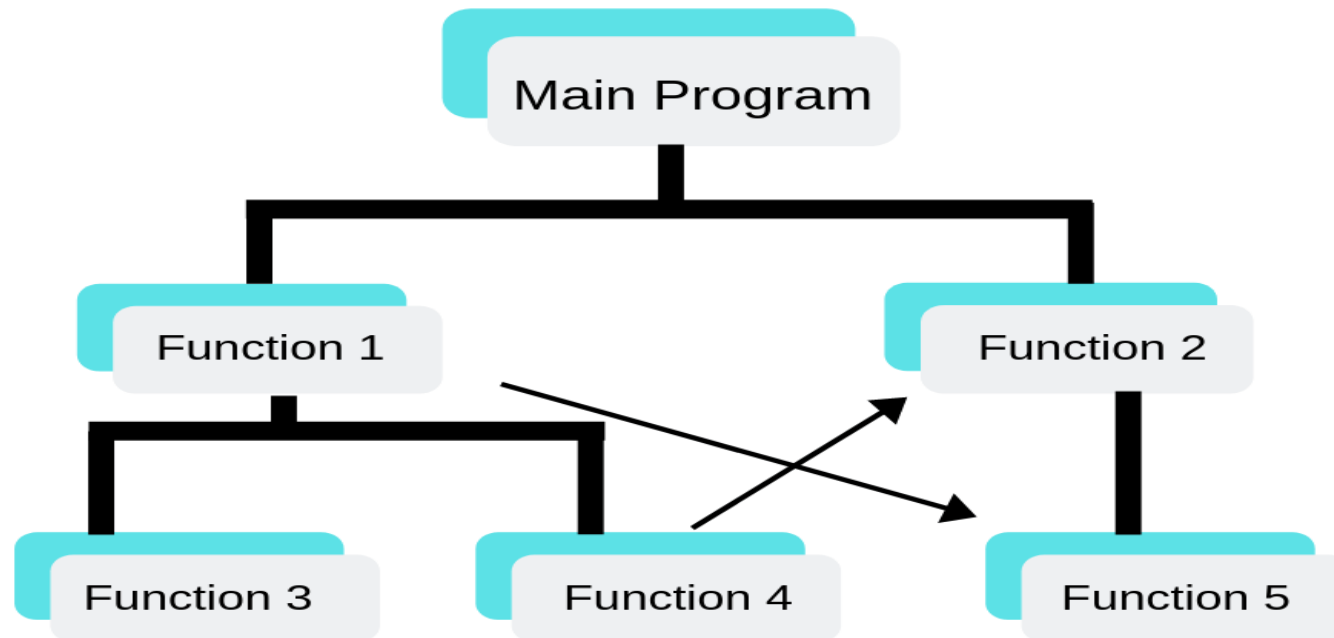
Benefits of C Programming

- Simple, versatile, general purpose language
- Robust language, which can be used to write any complex program.
- It has rich set of built-in functions and Operators
- Program execution are fast and efficient
- Well suited for writing both system software and business applications
- Highly portable
- Well suited for structured programming
- Dynamic memory allocation



Structured Programming Approach

- Generally the program is represented as a logical structure.
- C is called a structured programming language because it solves a large problem by dividing the problem into smaller structural blocks each of which handles a particular responsibility.



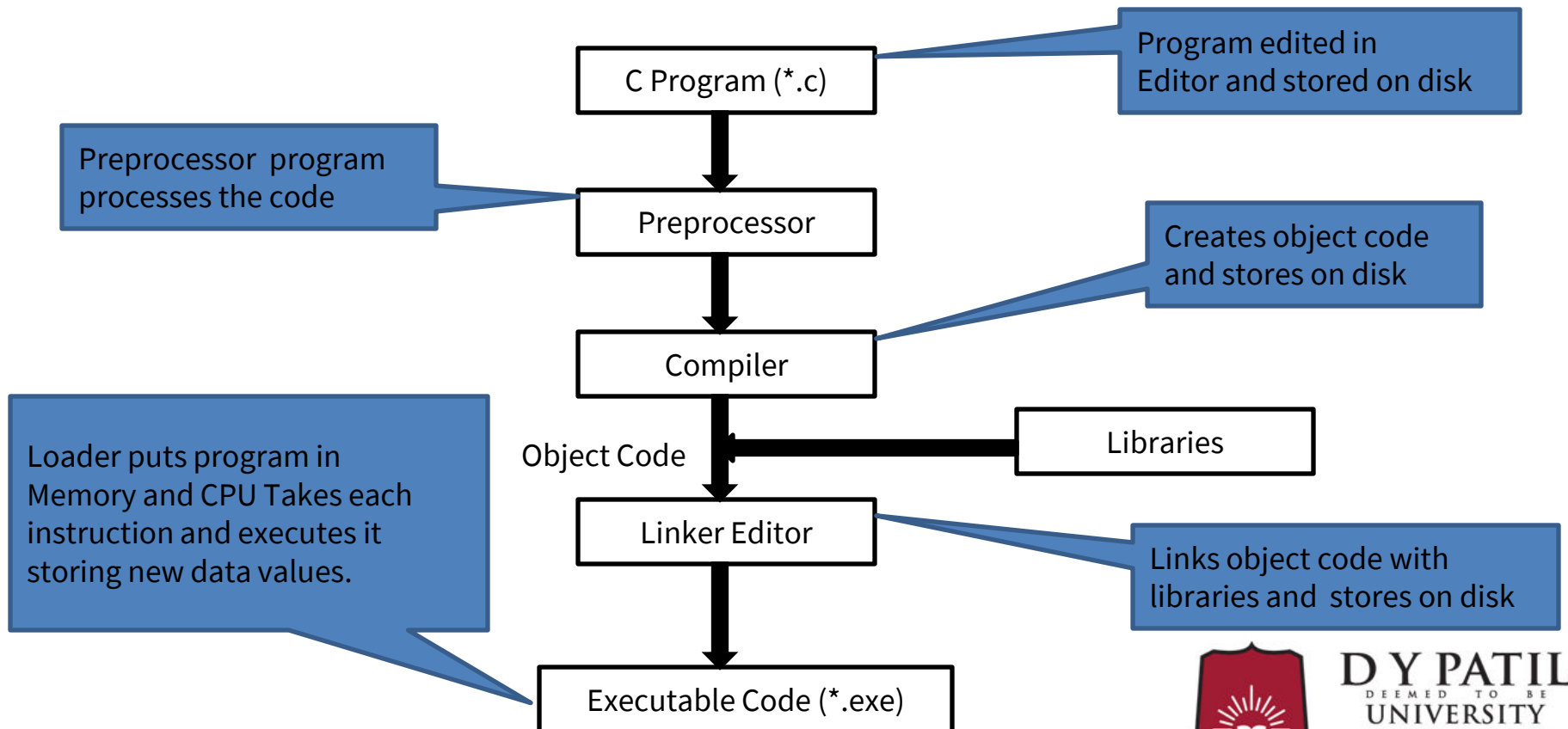
Structured Programming Approach (cont..)

- These structural blocks are –
 - **Decision making blocks** like if-else, nested if-else, switch-cases
 - **Repetitive blocks** like For-loop, While-loop, Do-while loop etc.
 - **Subroutines/Procedures** – functions
- Structured programming is simple and easy to understand and implement.
- Programmers do not require to know complex design concepts to start a new program.
- Easier to Debug.



Stages of Compilation

- Like most high level languages C also uses compiler to convert its source code (files with the extension .c) to object code (files with the extension .obj).
- The object code will be link-edited by the linker to form the machine language also known as executable codes (files with the extension .exe).



Structure of C Program

- A **C program** is divided into different sections as explained below:
 1. Documentation Section
 2. Header File Section
 3. Definition Section
 4. Global Declaration Section
 5. Main Section
 6. Sub Program Section



Structure of C Program (cont..)

- 1. Documentation Section :** The documentation section is the part of the program where the programmer gives the details associated with the program like name of the program, the details of the author , description etc.

Example :

```
/* Documentation Section */ ← Multiline comment
// File      : Addition.c
// Description : Addition of two numbers } Single Line
// Author    : Student                  } comment
```

- 2. Header File Section :** This part of the code is used to declare all the header files that will be used in the program. Compiler links the header files to the system libraries.

Example :

```
#include<stdio.h>
#include<conio.h>
```



Structure of C Program (cont..)

3. Definition Section : In this section, we define different constants. The keyword `define` is used in this part.

Example :

```
#define PI=3.14
```

4. Global Declaration Section : All the global variable used are declared in this part. The user-defined functions are also declared in this part of the code.

Example :

```
float area(float r);
```

```
int a=7;
```



Structure of C Program (cont..)

- 5. Main Section :** Each main function contains 2 parts. A declaration part and an Execution part. The declaration part is the part where all the variables are declared. The execution part has actual logic.

Example :

```
/*main() section */
int main()
{
    /* Declaration part */
    int a, b, sum;
    /* Execution part */
    printf("Enter two numbers:");
    scanf("%d%d", &a, &b);
    sum= a+b;
    printf("Sum=%d", sum);
}
```



Structure of C Program (cont..)

6. Sub Program Section : All the user-defined functions are defined in this section of the program.

Example :

```
int add(int a, int b)
{
    int d;
    d=a+b;
    return d;
}
```



C Preprocessor

- The **C Preprocessor** is not a part of the compiler, but is a separate step in the compilation process.
- C Preprocessor is just a text substitution tool and it instructs the compiler to do required pre-processing before the actual compilation.
- All preprocessor commands begin with a hash symbol (#).
- It must be the first nonblank character, and for readability, a preprocessor directive should begin in the first column.
- **For example :**

include<header_filename>

→ The **#include** directive will get a header file from **System Libraries** and add the text to the current source file.



Simple C Program

```
/* A first C Program*/
```

Description of program in multiline comment

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    printf("Hello
```

```
    Students");
```

```
}
```

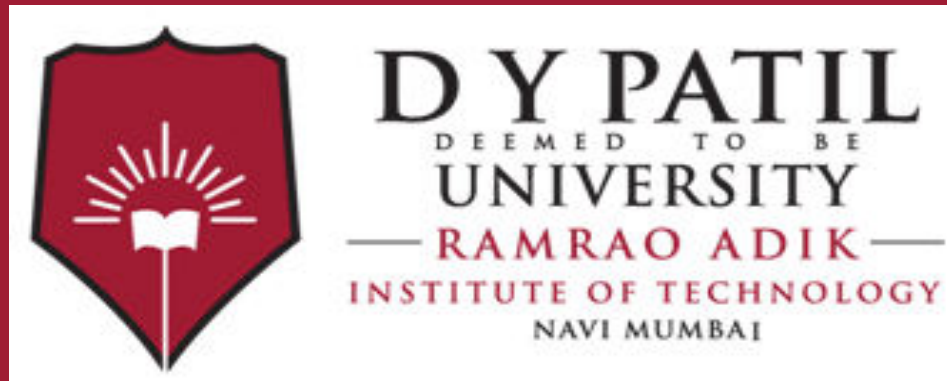
Declares the **main function**. The **"void"** specifies the **return type of main**. In this case, nothing is returned to the operating system.

The directive **#include** tells the preprocessor to include code from the file **stdio.h**. **stdio.h header file** contains declarations for functions that the program needs to use. A declaration for the **printf** function is in this file.

"{" denotes the start and **"}"** denote end of the program.

printf is a function from a standard C library that is used to print strings on the standard output device i.e. Monitor.





Thank You